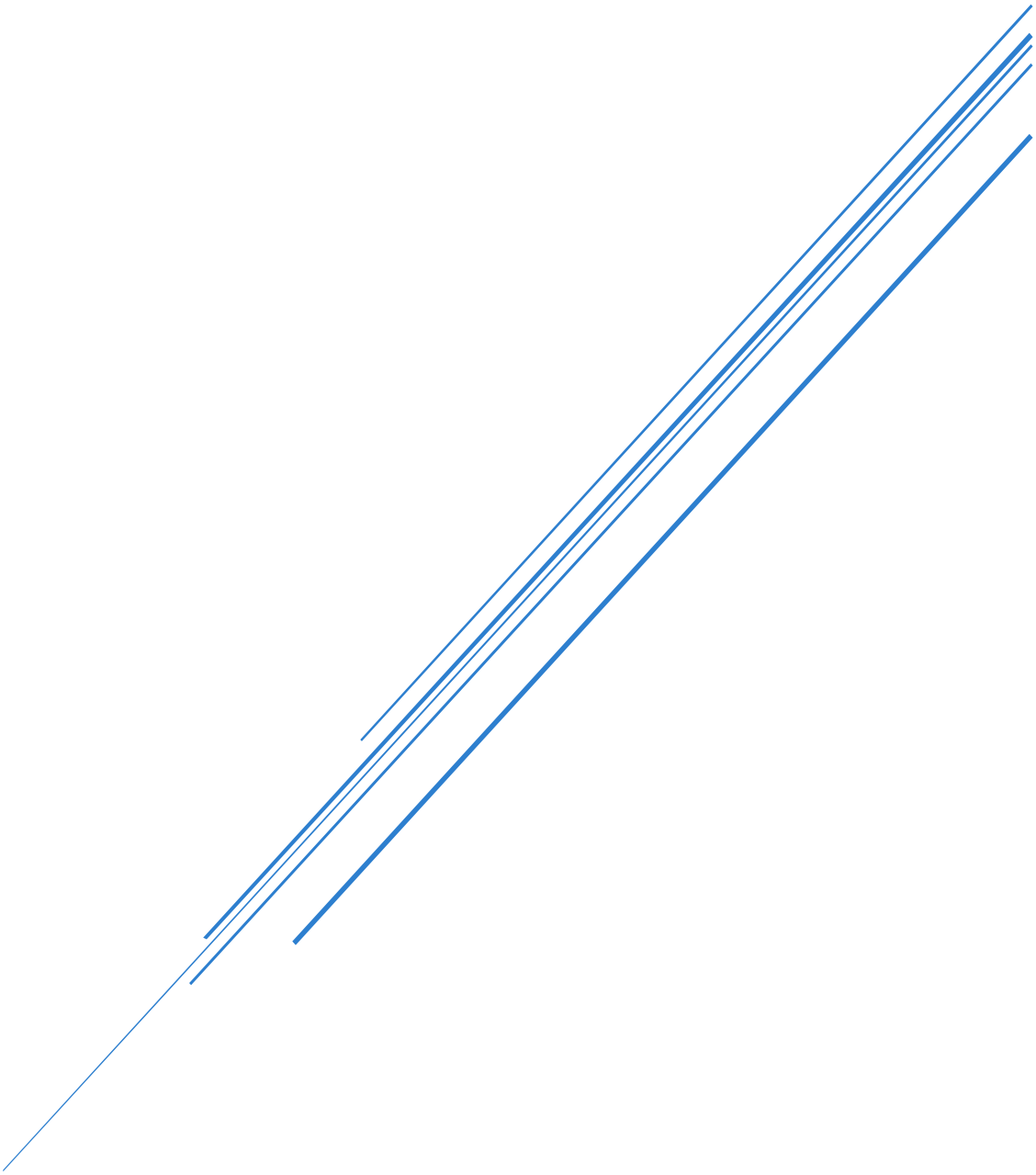
# CLDV6212 - PART 2

Group 7:
Thamsanqa Nkosi Ncube - ST10482062
Kandyce Smit - ST10467645
Kristen Keve - ST10472683



Lecturer: Farshida Wania
Module Code: CLDV6212

# Table of Contents

YouTube, GitHub and Docker Hub links: ..... 2
YouTube Link: ..... 2
GitHub Link: ..... 2
Docker Hub: ..... 2
TEAM MEMBER CONTRIBUTIONS ..... 3
FEATURE DESCRIPTIONS ..... 4
Order Queue Producer (Kandyce – ST10467645) ..... 4
Queue Trigger and Order Tracking (Kristen - ST10472683) ..... 4
Docker Compose Orchestration (Thami - ST10482062) ..... 4
POISON QUEUE ..... 6
DOCKER COMPOSE EXPLAINED ..... 8
Queue Producer (Kandyce - ST10467645) ..... 10
Queue Trigger and Order Tracking (Kristen - ST10472683) ..... 20
Docker Compose and Docker Hub (Thami) ..... 31
GitHub Commit History / Contributors ..... 36
GitHub Pull Requests Merged: ..... 36
TESTING SUMMARY ..... 39
DISCLOSURE ..... 40
Image credits ..... 40
CODE ATTRIBUTION REGISTER ..... 41
REFERENCES ..... 47

# YouTube, GitHub and Docker Hub links:

## YouTube Link:

https://youtu.be/iP3w9mEHbPM

## GitHub Link:

- PUBLIC REPOSITORY: https://github.com/ST10467645/CLDV6212_Part-2-g1-CoffeeNChill-ArcGroup7.git
- PRIVATE REPOSITORY (given by Emeris -Classroom 50-): https://github.com/EMGPSD/Group_1_CLDV6212_Part-2-_Group_7_on_ARC.git

## Docker Hub:

- coffeenchill-azurite: st10482062/coffeenchill-azurite - Docker Image
- coffeenchill-functions: st10482062/coffeenchill-functions - Docker Image

## *** Repository note ***

The original repository on GitHub from Classroom 50 is private; thus, just like in Part 1, the complete history of the branches and the commits were cloned into the public repository above for assessment purposes as the POE stated that the repository must be public. This clone includes all branches, all commits with the original authors and dates, as well as the merge commits.

# TEAM MEMBER CONTRIBUTIONS

| Name | Student Number | Role | What they built |
|---|---|---|---|
| Kandyce Smit | ST10467645 | Queue Producer, Postman & Documentation Lead | SubmitOrder queue producer (validation, JSON, Base64 encoding, timestamp, 202/400/503), GetOrders, GetOrderById, GetOrderStatus, DTO classes and mappers, host.json queue settings, sample JSON payloads, Part 2 Postman collection, README and this document |
| Kristen Keve | ST10472683 | Queue Trigger & Order Tracking Lead | ProcessOrderQueue (Base64 decoding, Orders table writes, Received > Preparing > Ready > Collected with 30 second waits), UpdateOrderStatus, poison queue handling and demonstration, Staff Document functions (Part 1) |
| Thamsanqa Ncube | ST10482062 | Docker Compose & Docker Hub Lead | Dockerfile, docker-compose.yml, Docker Hub v2.0 images, custom bridge network, Azurite data volume, Docker Postman checks, docker-commands.md |

## Equal contribution:

We divided Part 2 into three parts equally, which include the queue producer, the queue triggered function with order tracking and Docker Compose with Docker Hub. Every individual had ownership of one of these tasks and committed their work on a different branch, and all of us were involved in the video presentation, testing, and documentation. Below is the count of commits from our repository history (Based on the repository history available at the time that this document was written).

| Member | Branch | Commits | Pull requests merged | Days with commits |
|---|---|---|---|---|
| Kandyce Smit (ST10467645) | kandyce-part2-producer | 30 | 10 | 10 |
| Kristen Keve (ST10472683) | kristen-part2-processor | 15 | 11 | 7 |
| Thamsanqa Ncube (ST10482062) | thami-part2-compose | 11 | 7 | 7 |

# FEATURE DESCRIPTIONS

## Order Queue Producer (Kandyce – ST10467645)

Order producer is the first stage of the asynchronous order processing process. SubmitOrder is an Azure Function that receives an order in JSON format via HTTP triggered at POST /api/orders/queue (Microsoft Learn, 2025b). The function first checks whether the body is a valid JSON structure, whether the customer's name is provided, whether at least one item is selected with non-empty SKU and the order cost is more than zero. In case of any issue, the function returns a 400 Bad Request with a list of all issues, so that the client can correct everything at once and nothing gets into the queue. If the client does not provide the order number and the timestamp, the server creates them. Then the order gets serialized to JSON string, converted to Base64 in the code and submitted to the order-processing-queue using Azure Storage Queues client library (Microsoft Learn, 2026q; Microsoft Learn, 2026w). Base64 is just an encoding, not encryption. It changes the way how data is represented but not how it is protected. The configuration of host.json file named messageEncoding is none, so the exact text of Base64 will be stored in the queue. If the connection to the queue fails, the function traps the exception (Microsoft Learn, 2026r), logs and returns a 503 Service Unavailable in a couple of seconds instead of causing the function to crash. On a successful connection, it immediately returns 202 Accepted since the processing of the request occurs asynchronously in the background (IIE, 2026). The functions GetOrders, GetOrderById, and GetOrderStatus query the Orders table and return a DTO object with 404 and 400 errors (Microsoft Learn, 2026c).

## Queue Trigger and Order Tracking (Kristen - ST10472683)

ProcessOrderQueue is the other half of the process flow which runs automatically whenever there is a message received in order-processing-queue (Microsoft Learn, 2026g). It decodes the Base64 message into JSON format (Microsoft Learn, 2026o; Microsoft Learn, 2026p), creates an Order object out of it (Microsoft Learn, 2026s) and saves an entry in the Orders table, with order date as PartitionKey, OrderId as RowKey and status as Received (Microsoft Learn, 2026c; Microsoft Learn, 2026k). It then simulates the barista life cycle by delaying for 30 seconds before changing the status (Microsoft Learn, 2026j; Microsoft Learn, 2026m; Microsoft Learn, 2026n) from Received to Preparing, from Preparing to Ready, from Ready to Collected, and updating the row in the table at each status change. The lecture pointed out that there could be a use of a timer in this case, but Task.Delay would serve the purpose without blocking the thread. There is also a PUT endpoint called UpdateOrderStatus which allows the staff to move the order from one status to another and sends a 400 if an invalid or skipped status is sent and 404 if the order is unknown (Microsoft Learn, 2026t). However, when there is no way to process the message, an exception is thrown and recorded, and therefore after 5 attempts, the runtime will move the message to the order-processing-queue-poison (Microsoft Learn, 2026l).

## Docker Compose Orchestration (Thami - ST10482062)

My contribution for Part 2 of the POE was the Docker Compose configuration. There is a docker-compose.yml file in the root of the repository that launches both the Azurite emulator and the Functions app with one docker compose up command (Docker, 2026f). Both services use the v2.0 version-tagged images fetched from Docker Hub on the account st10482062 without a build section,

so any clean machine with Docker installed will be able to run the entire solution. The two services utilize a custom bridge network named coffeenchill-net, meaning that the Functions container can communicate with Azurite through the service name azurite, not localhost (Docker, 2026e), while a named volume azurite-data is used to save queue and table data after removing the containers (Docker, 2026g). The images were built from the multi-stage Dockerfile, tagged v2.0 and pushed to Docker Hub (Docker, 2026c; Docker, 2026d; Docker, 2026h; Microsoft Learn, 2026b). The API is exposed on port 7071 to be able to test it with Postman when Visual Studio is not opened.

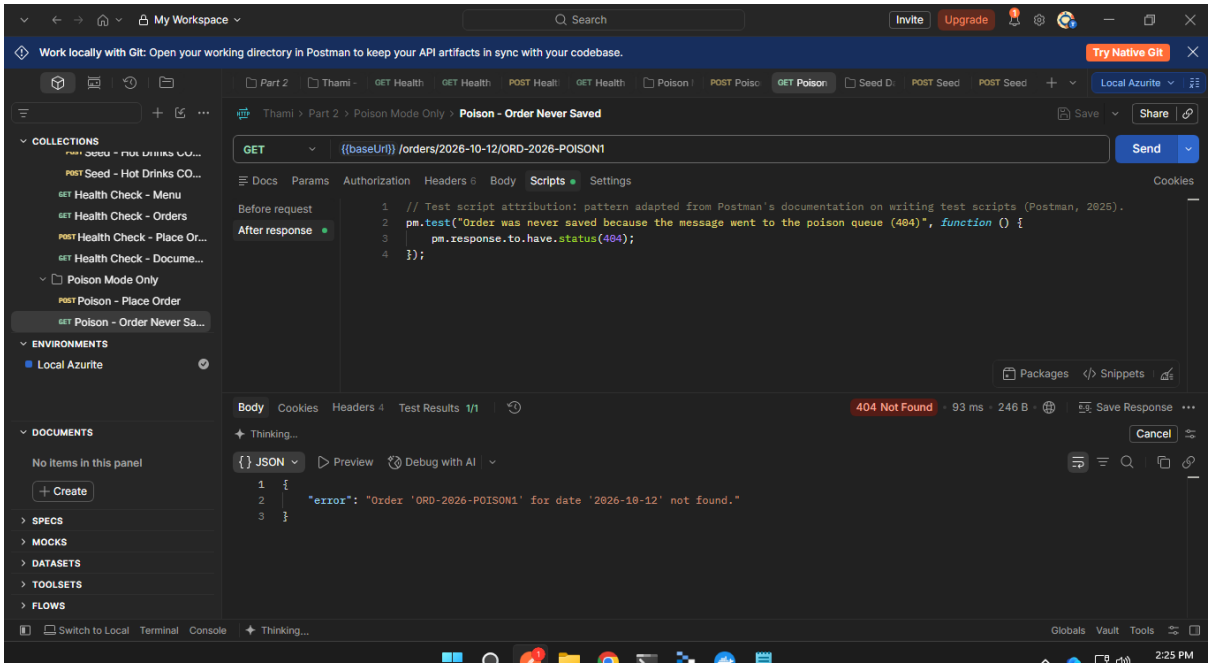
# POISON QUEUE

Poison queue is a storage area for messages that are unable to get processed successfully. Azure Functions creates it automatically; its name will be based on the name of the original queue: order-processing-queue-poison. In case if maxDequeueCount is equal to 5 and a certain message fails 5 times in a row, it will be moved to the poison queue without blocking it (Microsoft Learn, 2026l). Since our application is working properly, messages do not fail, and thus, there is no poison queue until the message fails for the first time. We put a deliberately wrong message ("this is not valid JSON or Base64") to the order-processing-queue using Azure Storage Explorer to make it work. SubmitOrder blocks bad data with 400 error, and therefore, it was put manually to test the mechanism provided by the trigger itself. It could not be decoded or read, was logged 5 times, and then, was sent to the poison queue. Figures 19 to 22 represent queue list before the failure, insertion of the wrong message, logging of the failed attempts, and poison queue with the message inside.

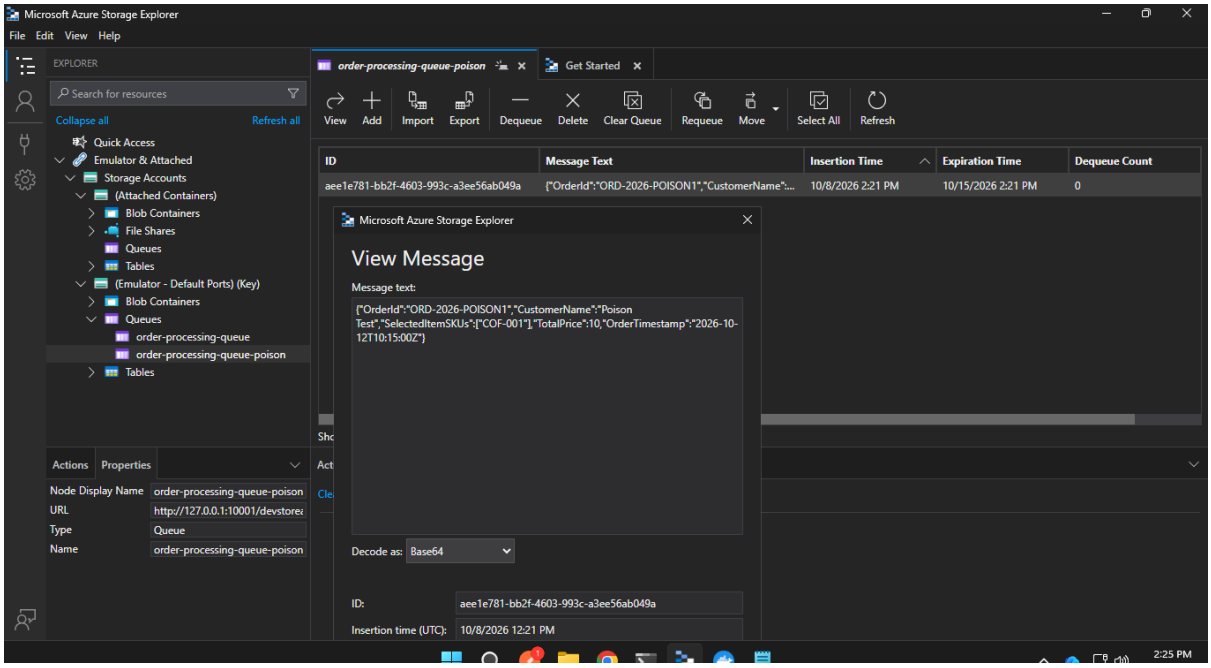
To demonstrate it we set ENABLE_POISON_TEST to true in docker-compose.yml, which makes the function throw on purpose and test examples can be seen below.

**Screenshots:**

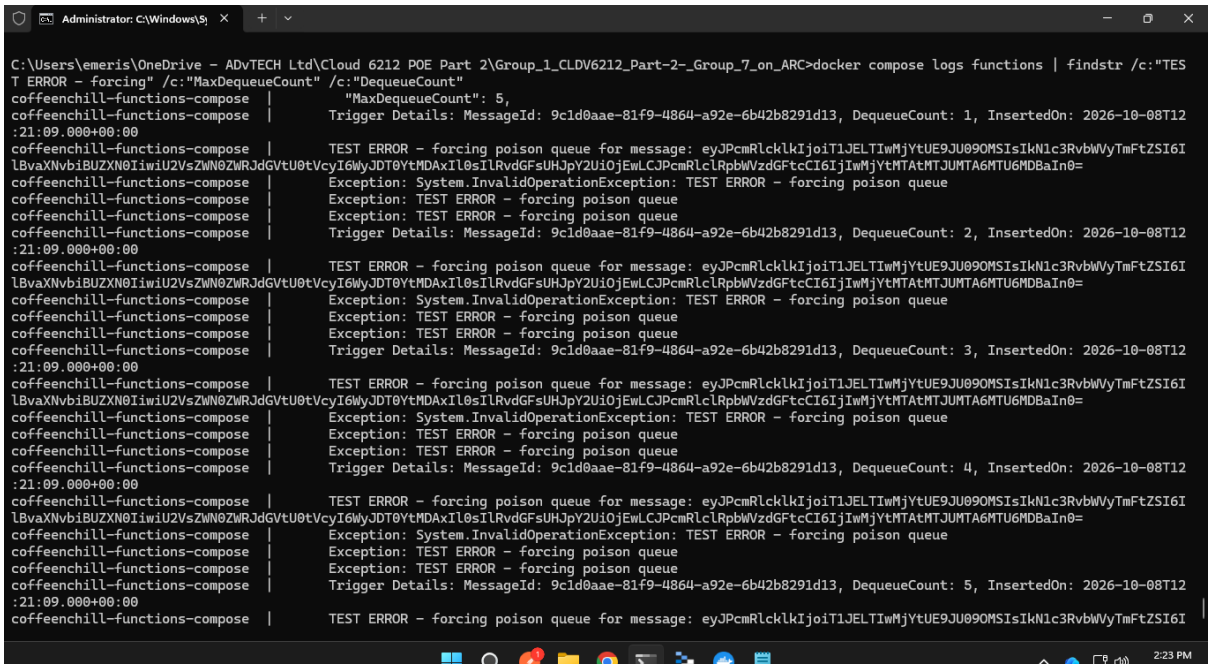
Poison order not saved:

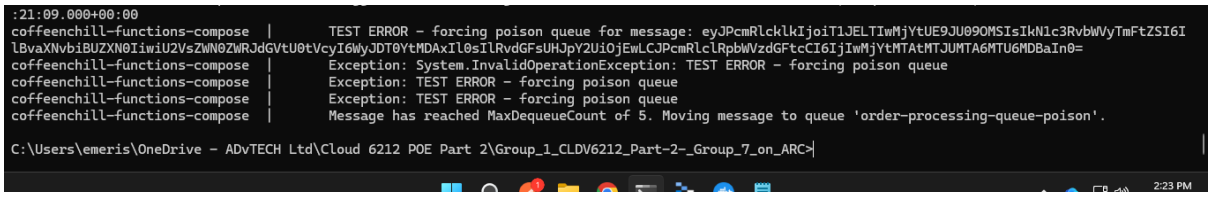



Poison queue message:

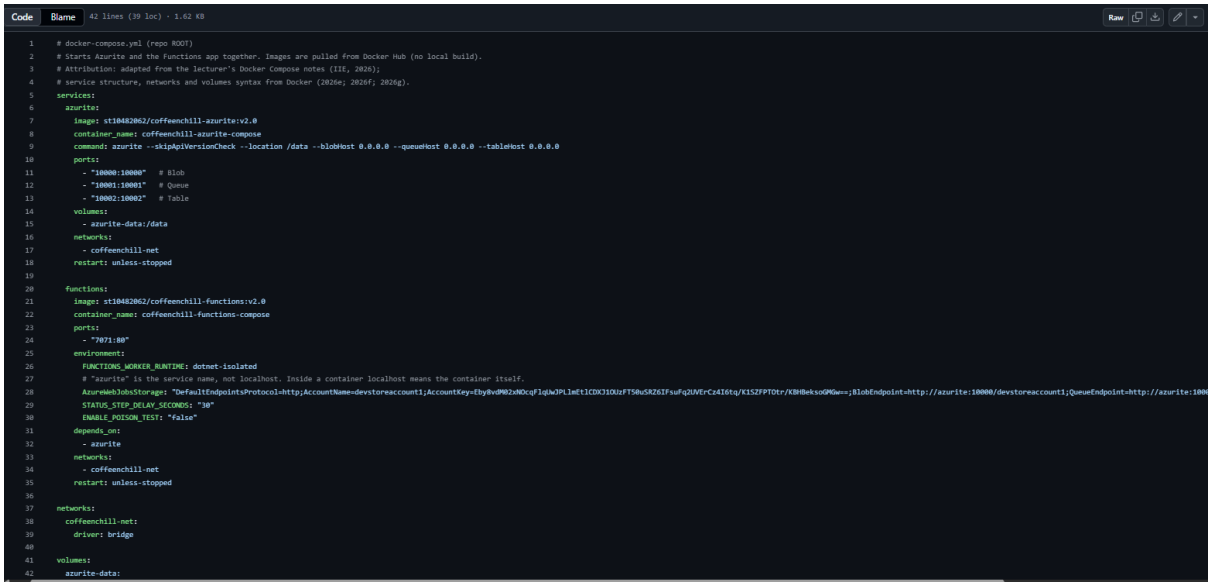



Poison logs in compose:



# DOCKER COMPOSE EXPLAINED




```yaml
# docker-compose.yml (repo ROOT)
# Starts Azurite and the Functions app together. Images are pulled from Docker Hub (no local build).
# Attribution: adapted from the lecturer's Docker Compose notes (IIE, 2026);
# service structure, networks and volumes syntax from Docker (2026e; 2026f; 2026g).
services:
  azurite:
    image: st10482062/coffeenchill-azurite:v2.0
    container_name: coffeenchill-azurite-compose
    command: azurite --skipApiVersionCheck --location /data --blobHost 0.0.0.0 --queueHost 0.0.0.0 --tableHost 0.0.0.0
    ports:
      - "10000:10000"   # Blob
      - "10001:10001"   # Queue
      - "10002:10002"   # Table
    volumes:
      - azurite-data:/data
    networks:
      - coffeenchill-net
    restart: unless-stopped

  functions:
    image: st10482062/coffeenchill-functions:v2.0
    container_name: coffeenchill-functions-compose
    ports:
      - "7071:80"
    environment:
      FUNCTIONS_WORKER_RUNTIME: dotnet-isolated
      # "azurite" is the service name, not localhost. Inside a container localhost means the container itself.
      AzureWebJobsStorage: "DefaultEndpointsProtocol=http;AccountName=devstoreaccount1;AccountKey=Eby8vdM02xNDcqFlqUwJPLlmEtlCDXJ1OUzFT50uSRZ6IFsuFq2UVErCz4I6tq/K1SZFPTOtr/KBHBeksoGMGw==;BlobEndpoint=http://azurite:10000/devstoreaccount1;QueueEndpoint=http://azurite:1000
      STATUS_STEP_DELAY_SECONDS: "30"
      ENABLE_POISON_TEST: "false"
    depends_on:
      - azurite
    networks:
      - coffeenchill-net
    restart: unless-stopped

networks:
  coffeenchill-net:
    driver: bridge

volumes:
  azurite-data:
```

| Setting | Meaning |
|---|---|
| image: st10482062/coffeenchill-functions:v2.0 | The application image and its exact version, pulled from Docker Hub |
| image: st10482062/coffeenchill-azurite:v2.0 | The storage emulator image, same version tag |
| (no build: section) | Nothing is built locally, everything is pulled from Docker Hub |
| ports 7071:80 | Computer port 7071 goes to port 80 inside the functions container |
| ports 10000, 10001, 10002 | Azurite blob, queue and table |
| networks: coffeenchill-net (bridge) | Our own network; the containers reach each other by service name |
| volumes: azurite-data:/data | Stored queue and table data survive docker compose down |
| AzureWebJobsStorage | Connection string that uses the host name azurite, not localhost |
| STATUS_STEP_DELAY_SECONDS: "30" | Seconds between order statuses |
| ENABLE_POISON_TEST: "false" | Optional switch that forces a failure for the poison queue demo |
| depends_on: azurite | Azurite Ensures the Azurite container starts before the Functions container but does not wait until Azurite is ready. |

| Setting | Meaning |
| --- | --- |
| restart: unless-stopped | Restarts the container if it stops or crashes, unless it has been explicitly stopped. |

**Commands used to reach docker compose up:**

## Docker Commands Reference — CoffeeNChill

| Task | Command |
| --- | --- |
| Start Azurite | `docker run -d --name coffeenchill-azurite -p 10000:10000 -p 10001:10001 -p 10002:10002 mcr.microsoft.com/azure-storage/azurite` |
| Build Functions image | `docker build -t st10482062/coffeenchill-functions:v1.0 .` |
| Push Functions image | `docker push st10482062/coffeenchill-functions:v1.0` |
| Run Functions container | `docker run -p 7071:80 -e AzureWebJobsStorage="UseDevelopmentStorage=true" st10482062/coffeenchill-functions:v1.0` |
| Re-tag Azurite image | `docker tag mcr.microsoft.com/azure-storage/azurite st10482062/coffeenchill-azurite:v1.0` |
| Push Azurite image | `docker push st10482062/coffeenchill-azurite:v1.0` |

### Part 2: Docker Compose and Docker Hub v2.0

#### Compose commands

```
docker compose config                 # check the file for errors
docker compose up                     # start Azurite and Functions (live logs)
docker compose up -d                  # start in the background
docker compose pull                   # download the latest v2.0 images (run after a rebuild)
docker compose ps                     # list the compose containers
docker compose logs functions         # show the Functions logs
docker compose logs functions | findstr /c:"Host.Functions."   # list the loaded functions (Windows)
docker compose down                   # stop and remove containers and network (keeps the volume)
docker compose down -v                # DANGER: also deletes the Azurite data volume
```

#### Network and volume checks

```
docker network ls
docker network inspect NETWORK_NAME
docker volume ls
```

# EVIDENCE SCREENSHOTS

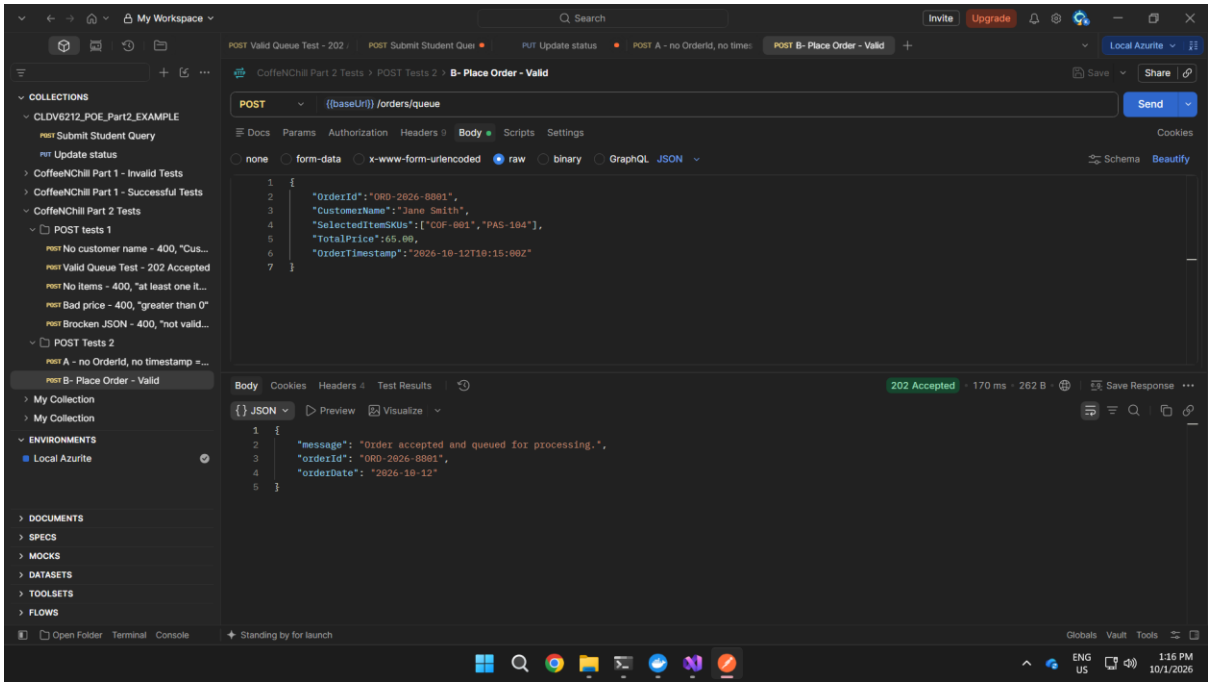
*** NOTE***

All these screenshots can be found in our GitHub repository under docs/screenshots_part2 and each members screenshots can be found in their own folders.

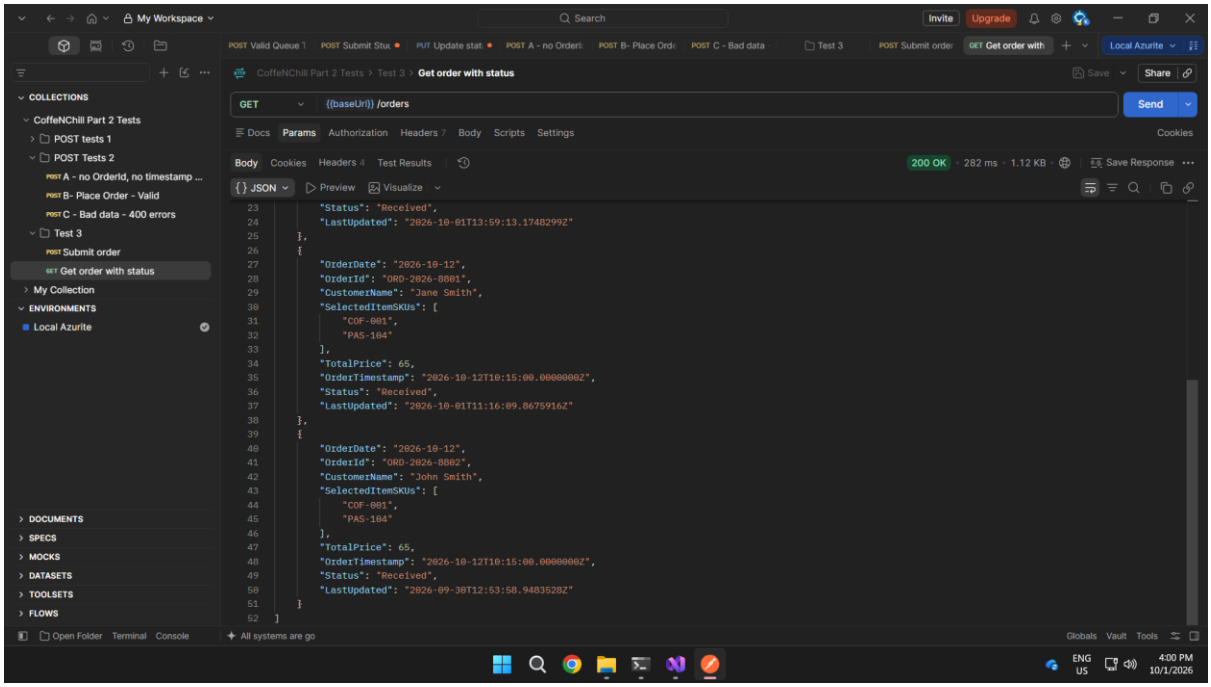
## Queue Producer (Kandyce - ST10467645)

### Valid orders and status:

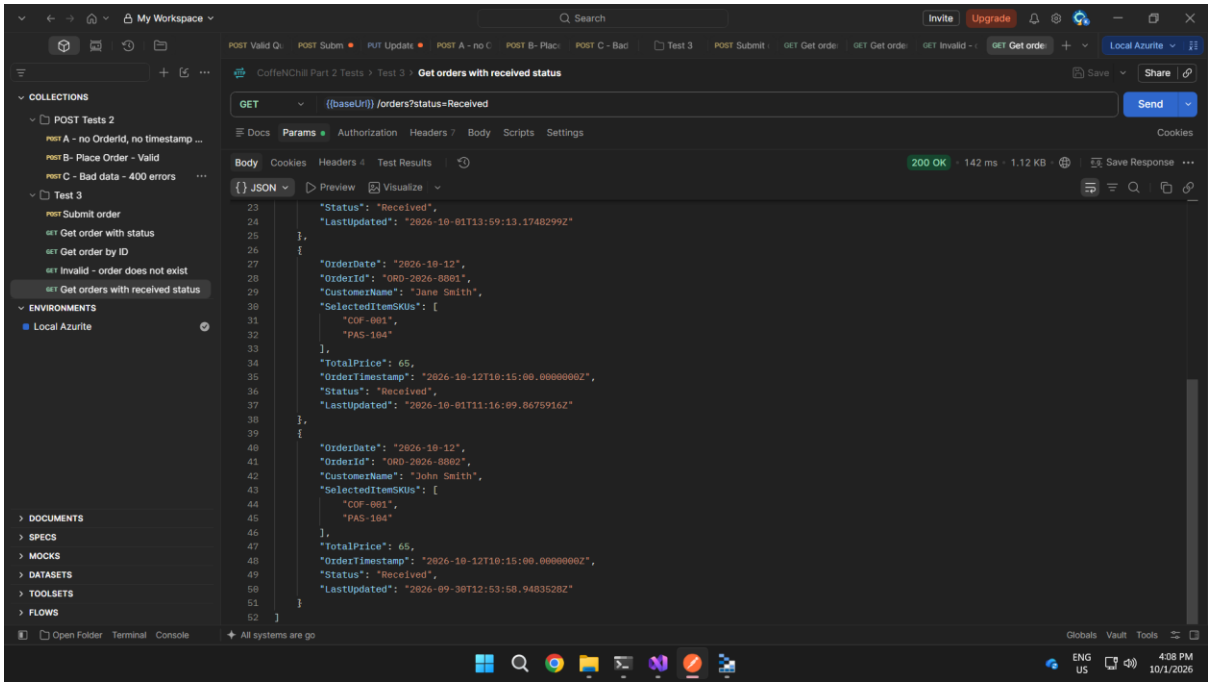
Valid order accepted - 202 (P2_K01_producer_202_accepted.png)



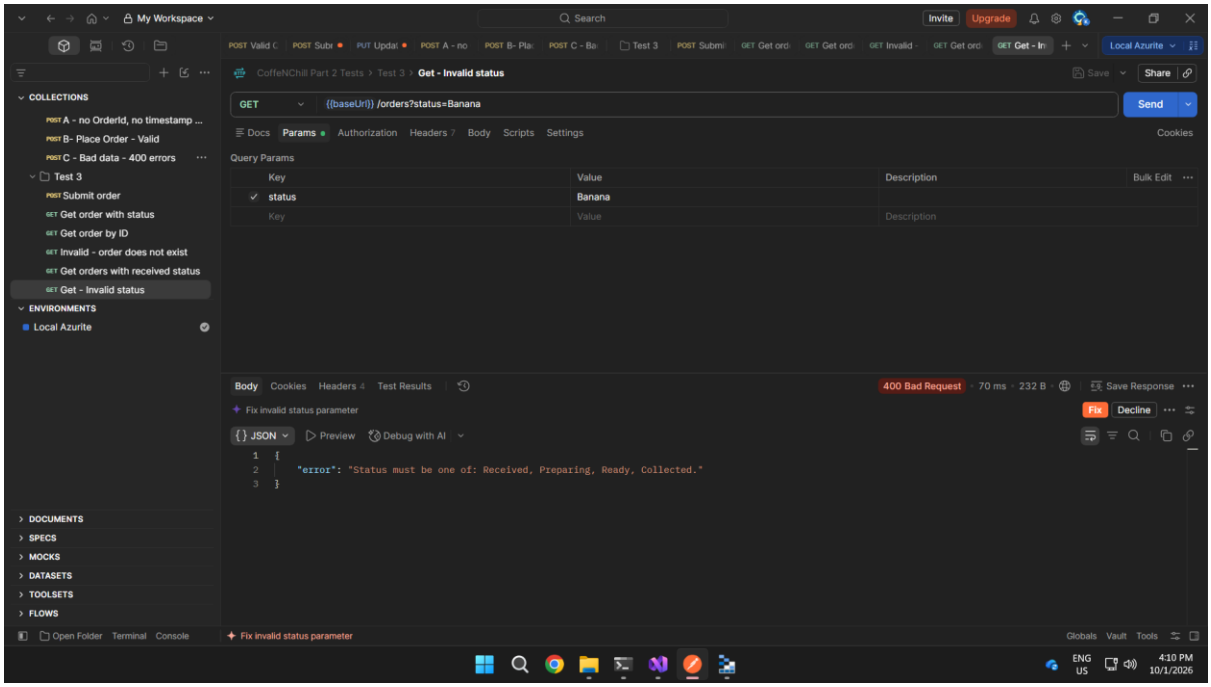
## Get all orders – 200 (P2_K05_get_all_orders_200.png)



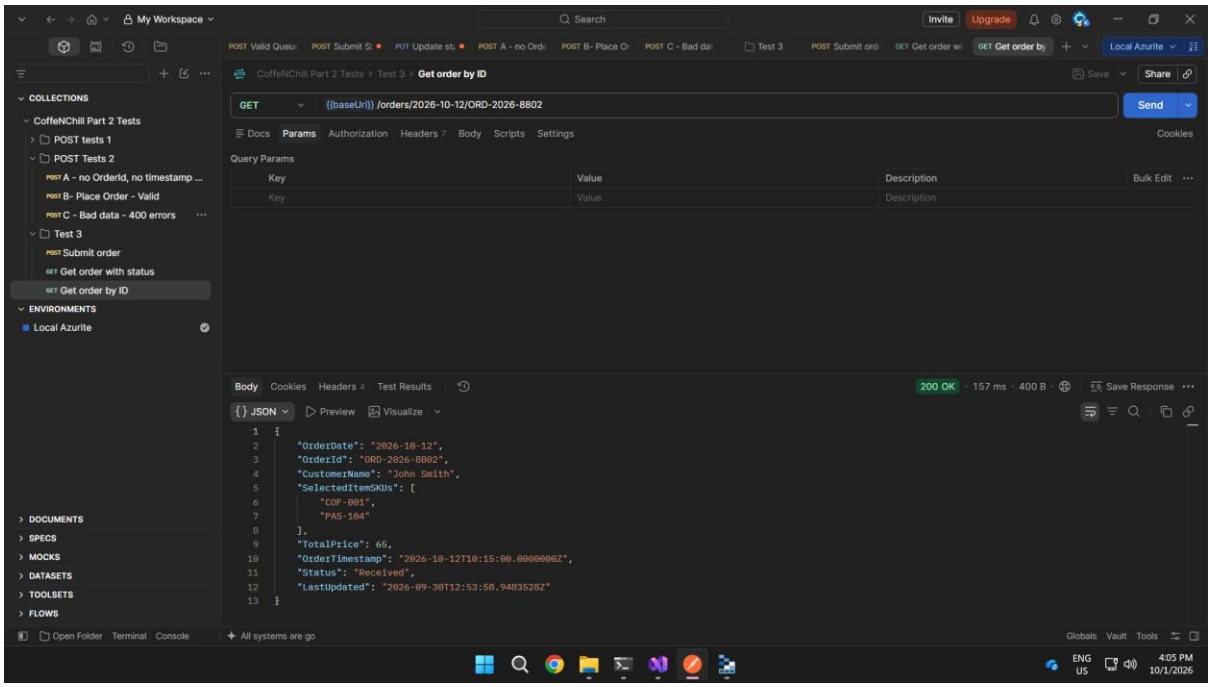

## Get all orders with a certain status, in this example status is “Received” (P2_K05_get_all_orders_recieved_200.png)



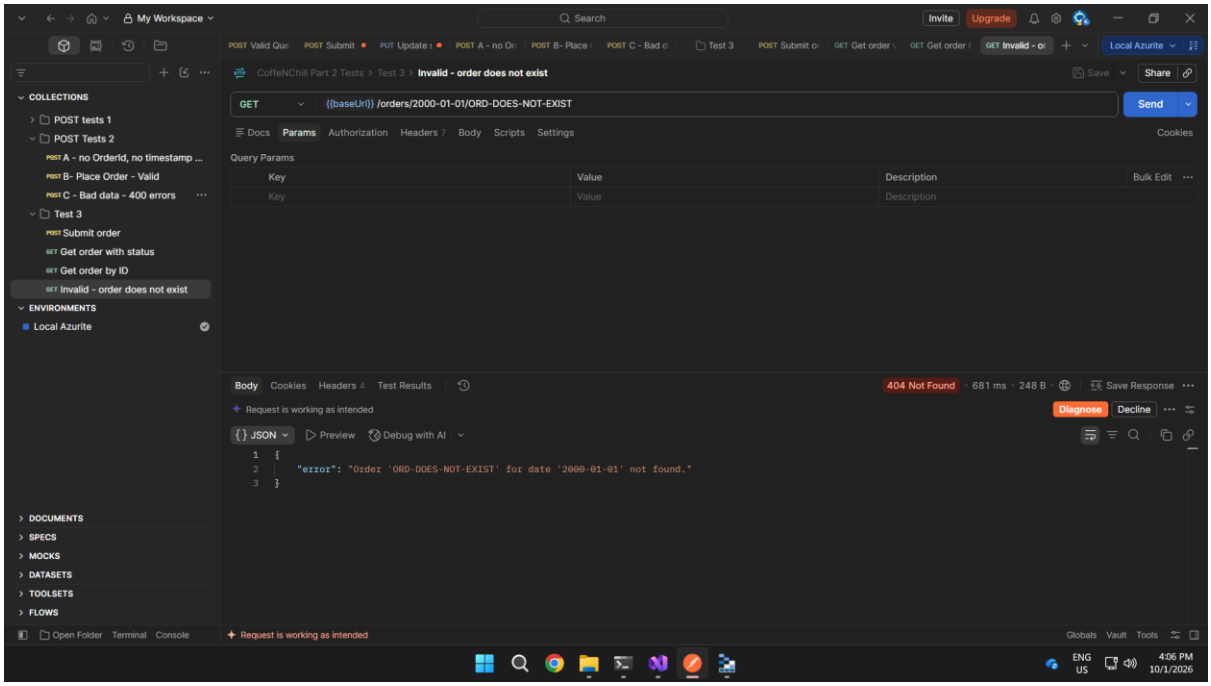
## Get an order using an INVALID status, in this example, “Bannana” (P2_K08_get_orders_invalid_status_400.png)



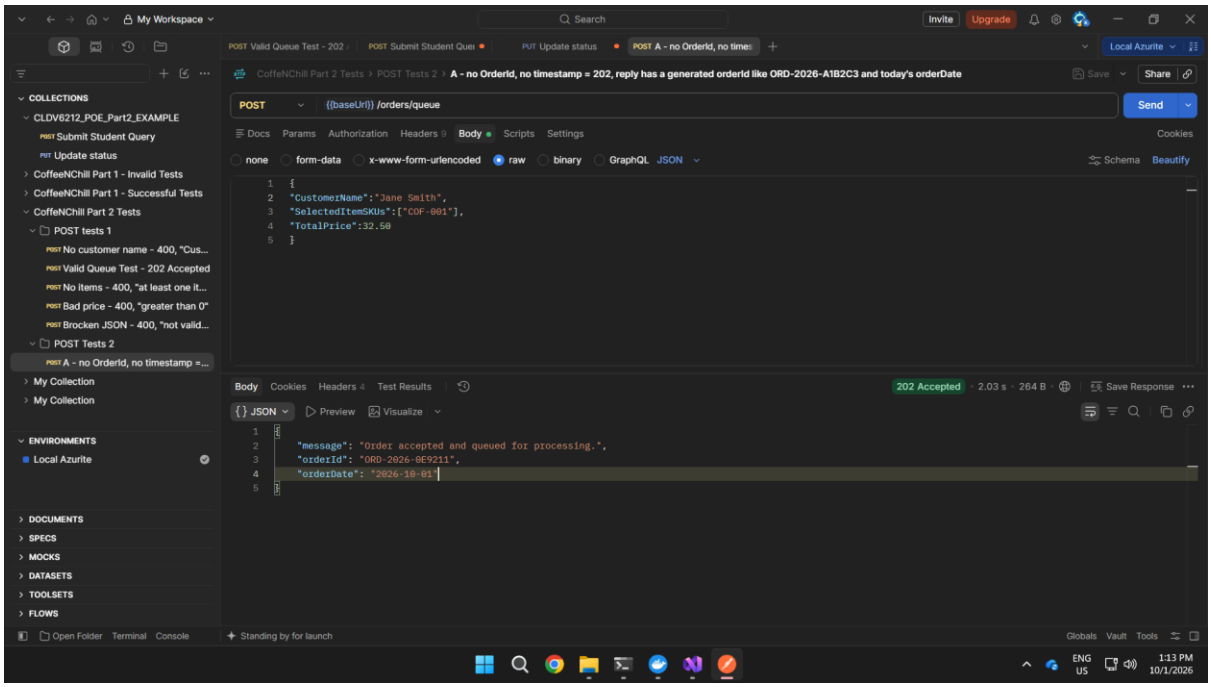

## Get an order by its ID – 200 (P2_K06_get_order_by_id_200.png)




## Order NOT found (P2_K07_get_order_not_found_404.png)

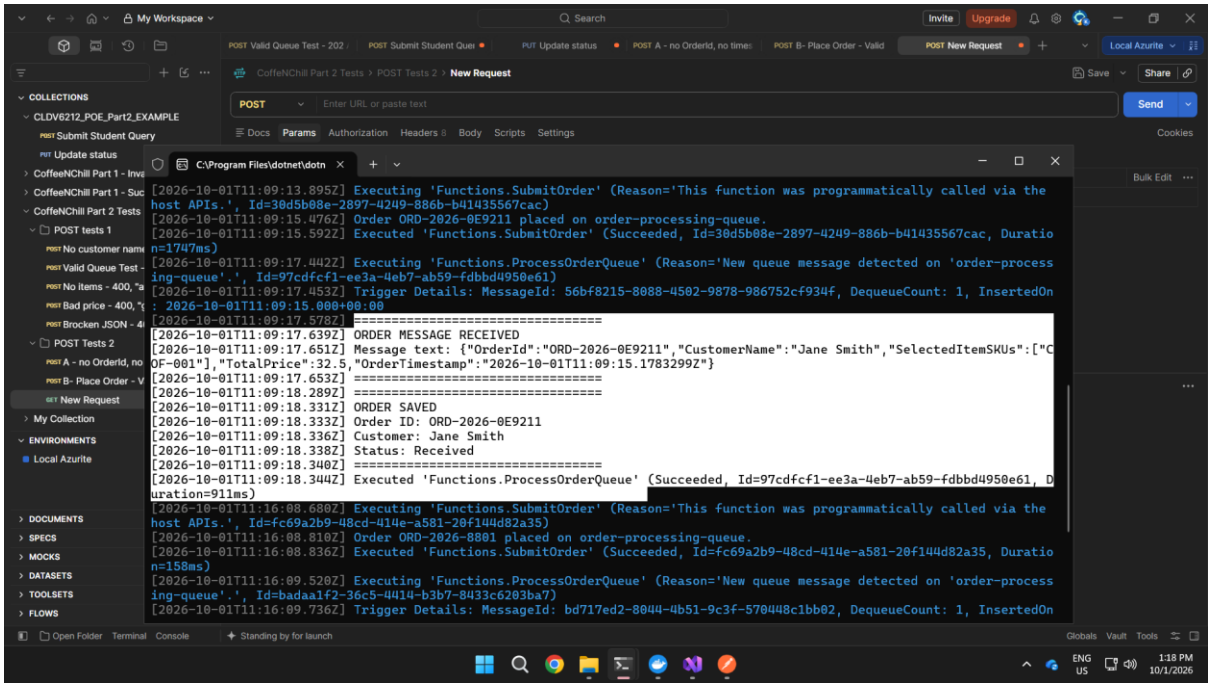



## Auto generated orderID and timestamp (P2_K02_generated_orderid_timestamp.png)

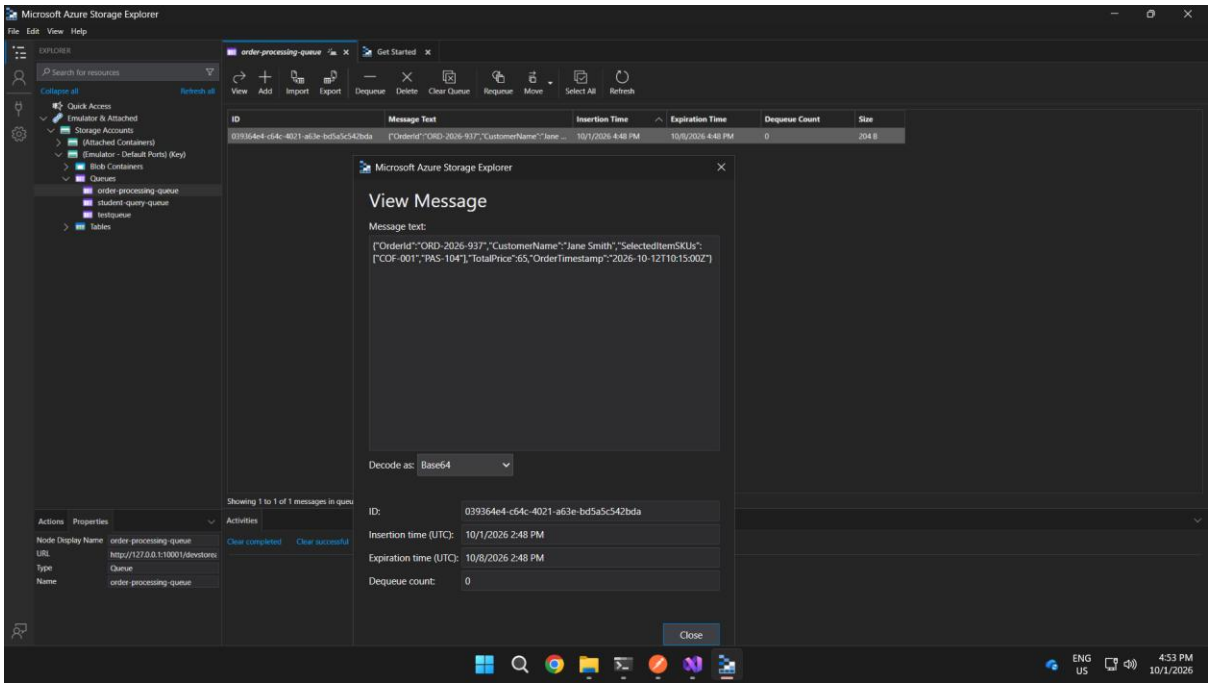



## Queue screenshots:

## Console output of order being queued (P2_K02b_console_order_queued.png)

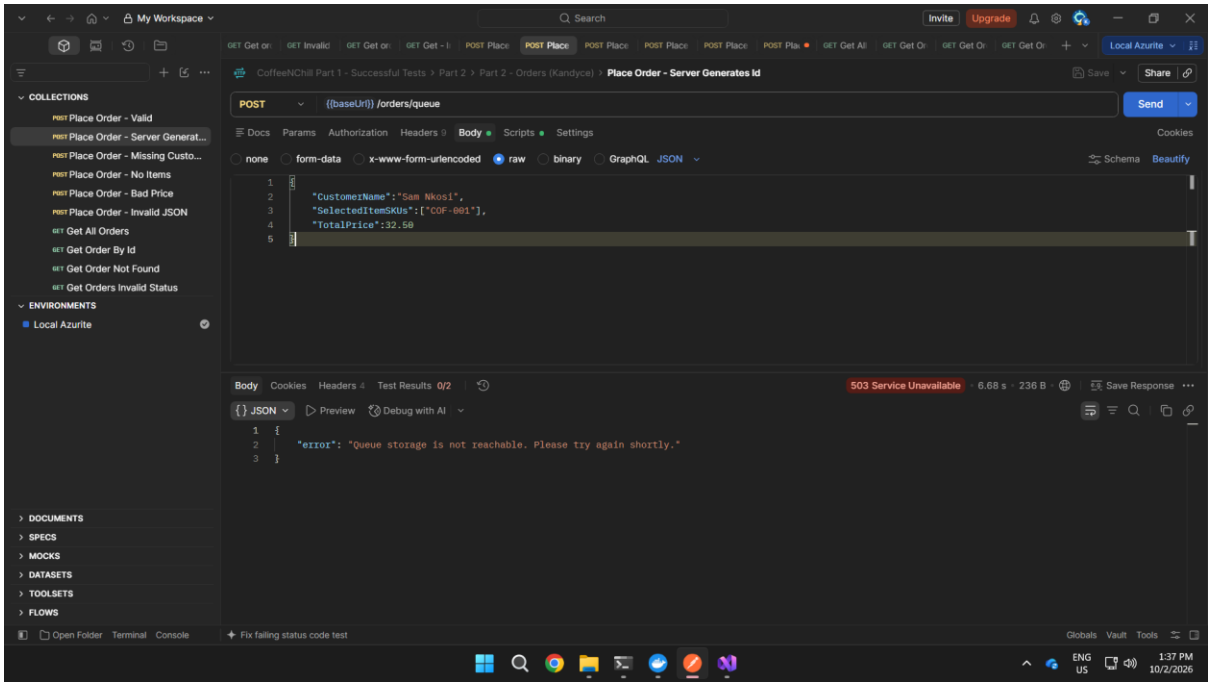



## Message waiting in the queue (P2_K11_message_waiting_in_queue.png)



## 503 error:

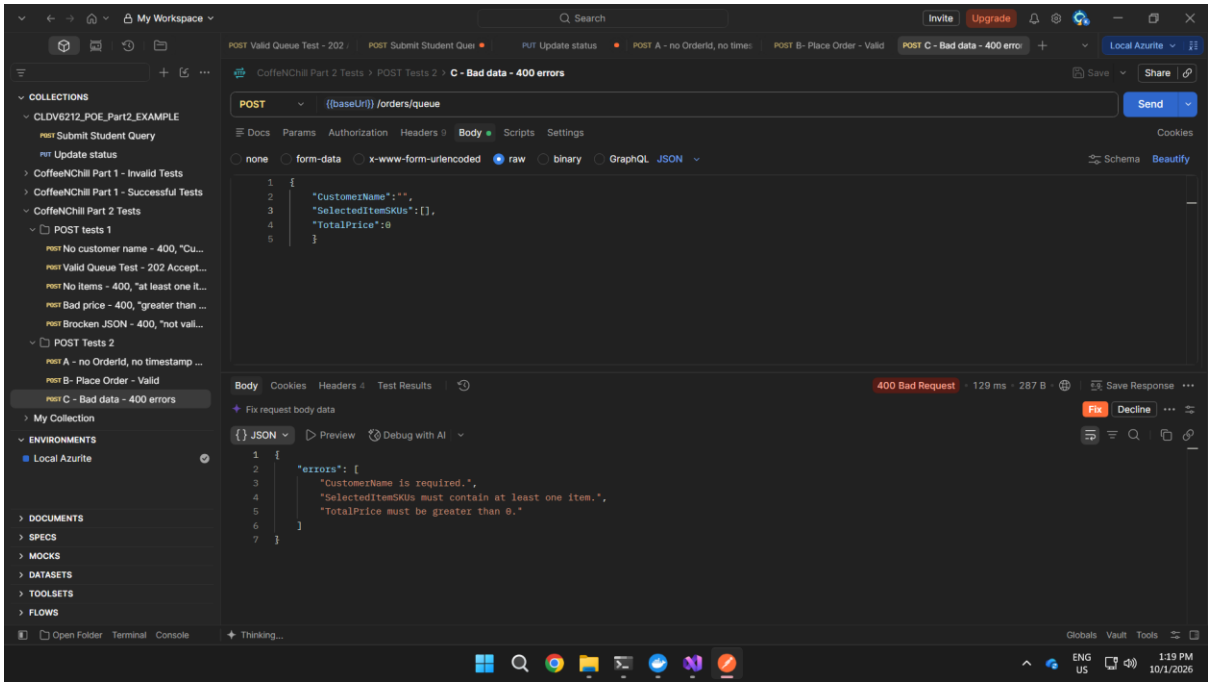
503 error when queue storage is unreachable (P2_K04_queue_unavailable_503.png)



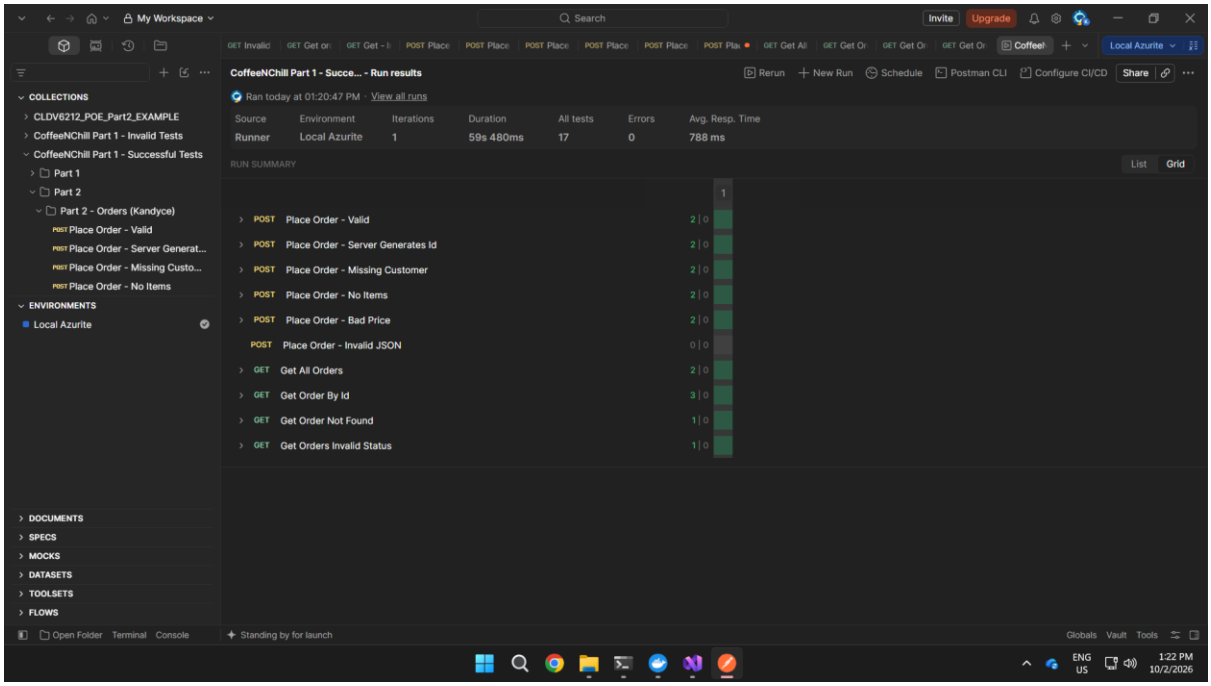

## Postman:

Postman invalid orders returning 400 with the errors (for example; missing name, empty SKUs, price 0):

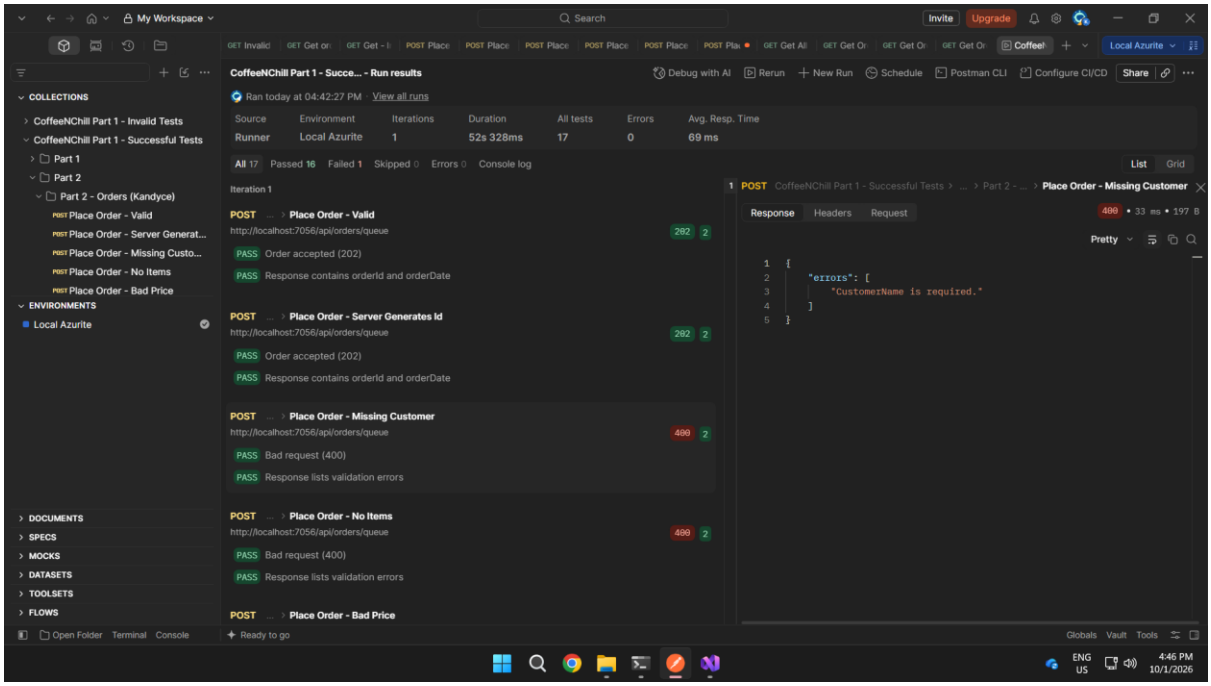
No customer name (P2_K03_validation_400_errors.png)



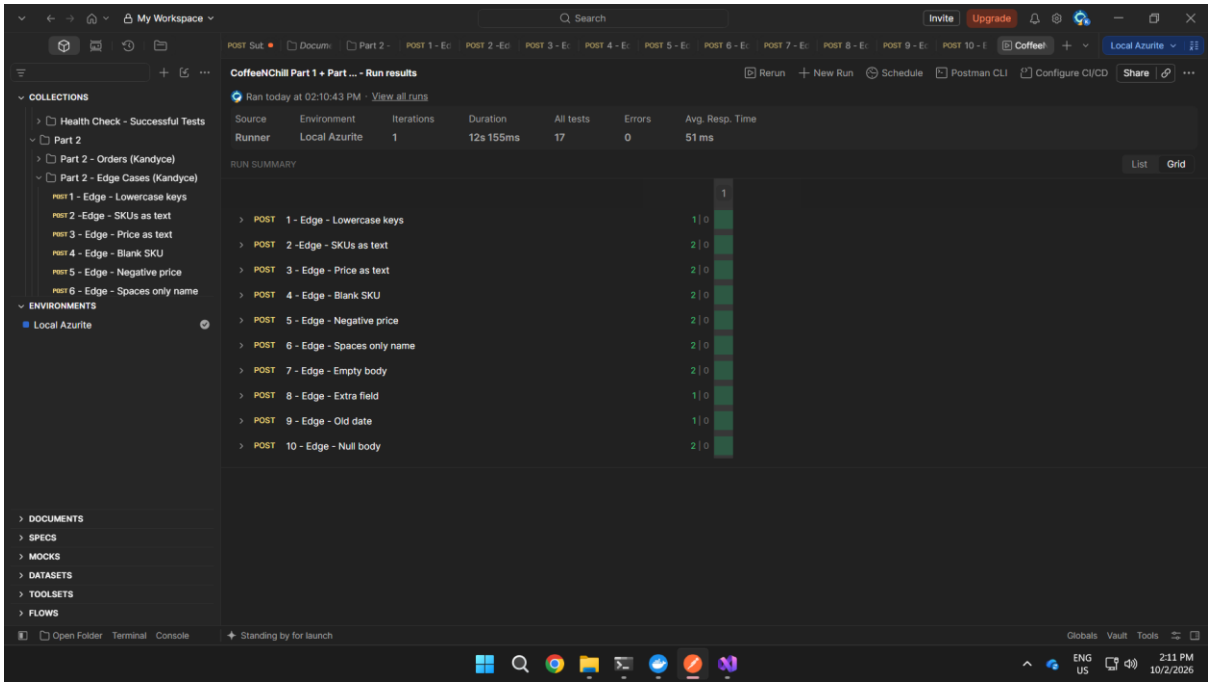

## My tests run all green (P2_K09_postman_folder_run_green.png)



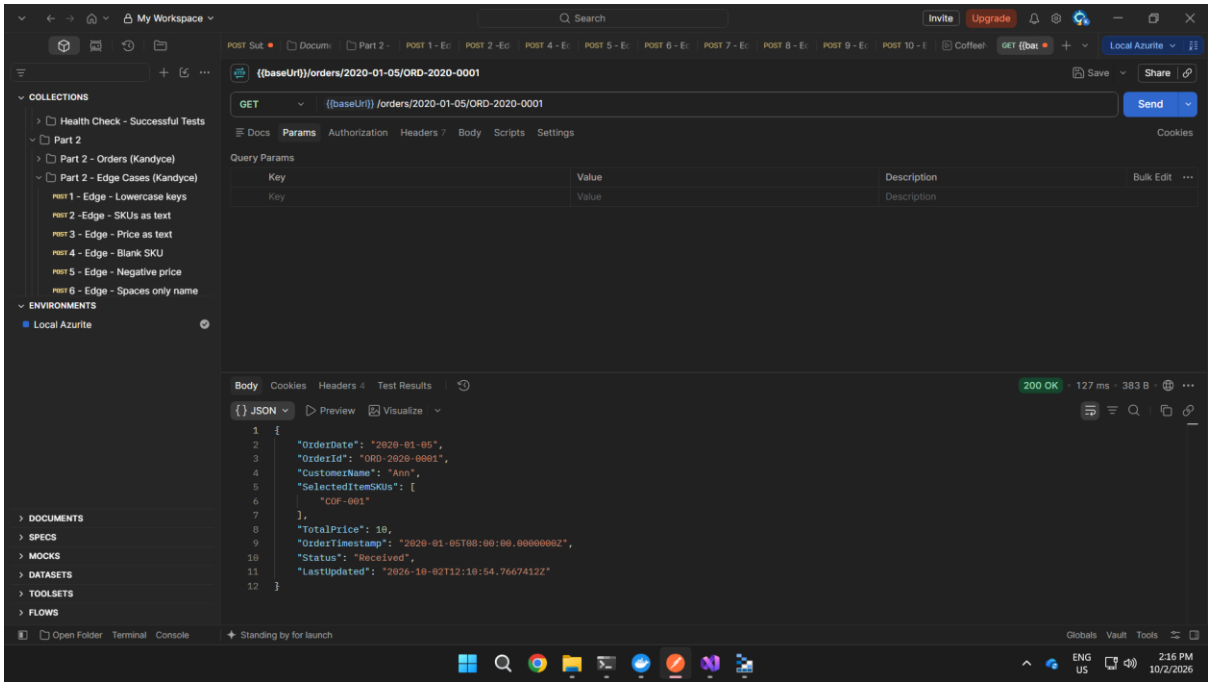

## Tab results of the successful tests (P2_K10_postman_test_results_tab.png)



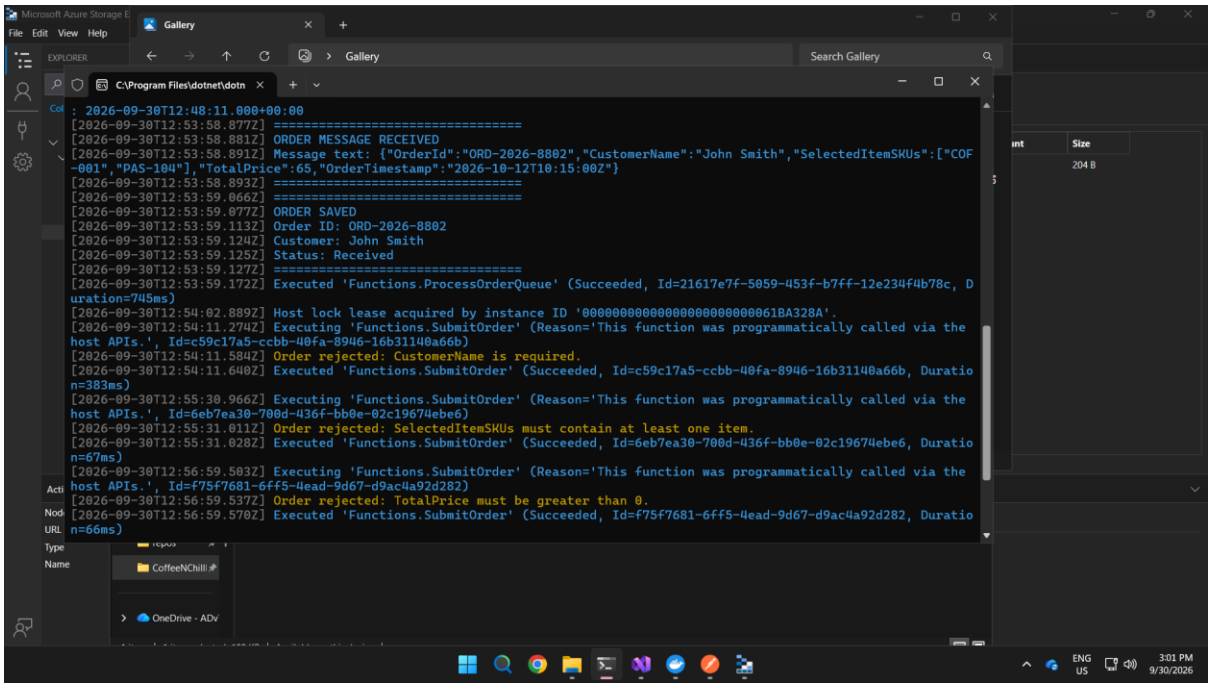

Edge cases successful run (P2_K13_postman_edge_cases_run.png.png)



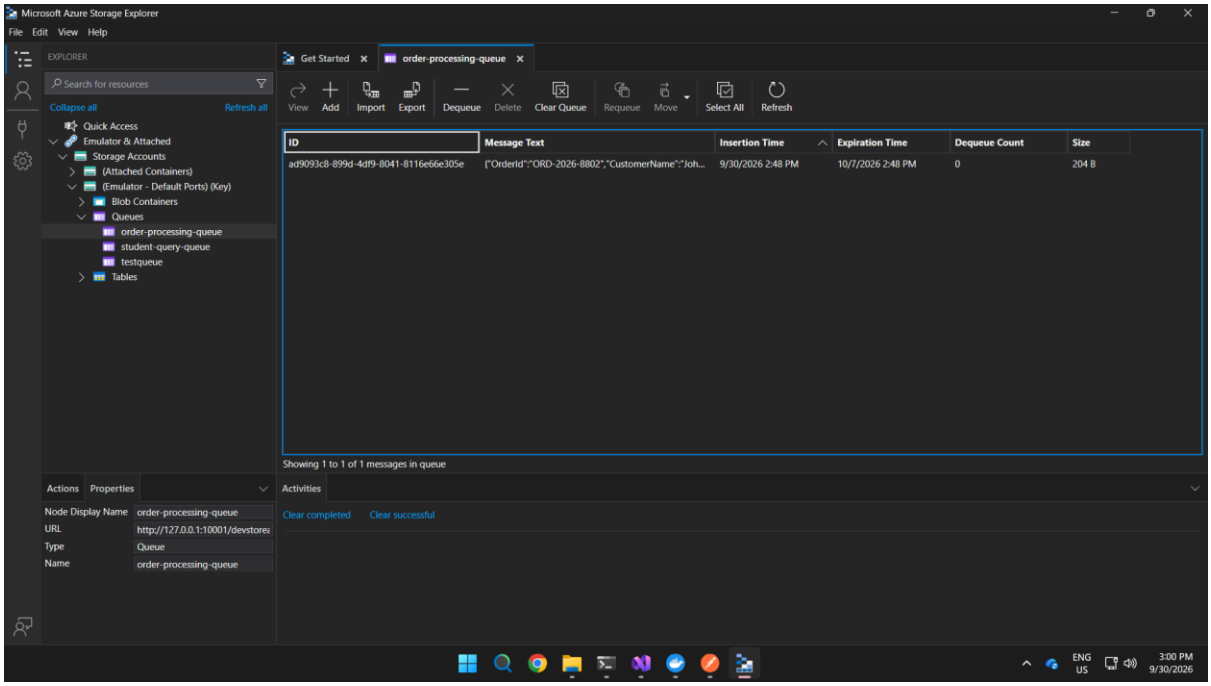

Finding an order from an old date
(P2_K14_old_date_order_found_200.png)




## Showing Postman response in the terminal (PostManTestsResponsesInTerminal.png)



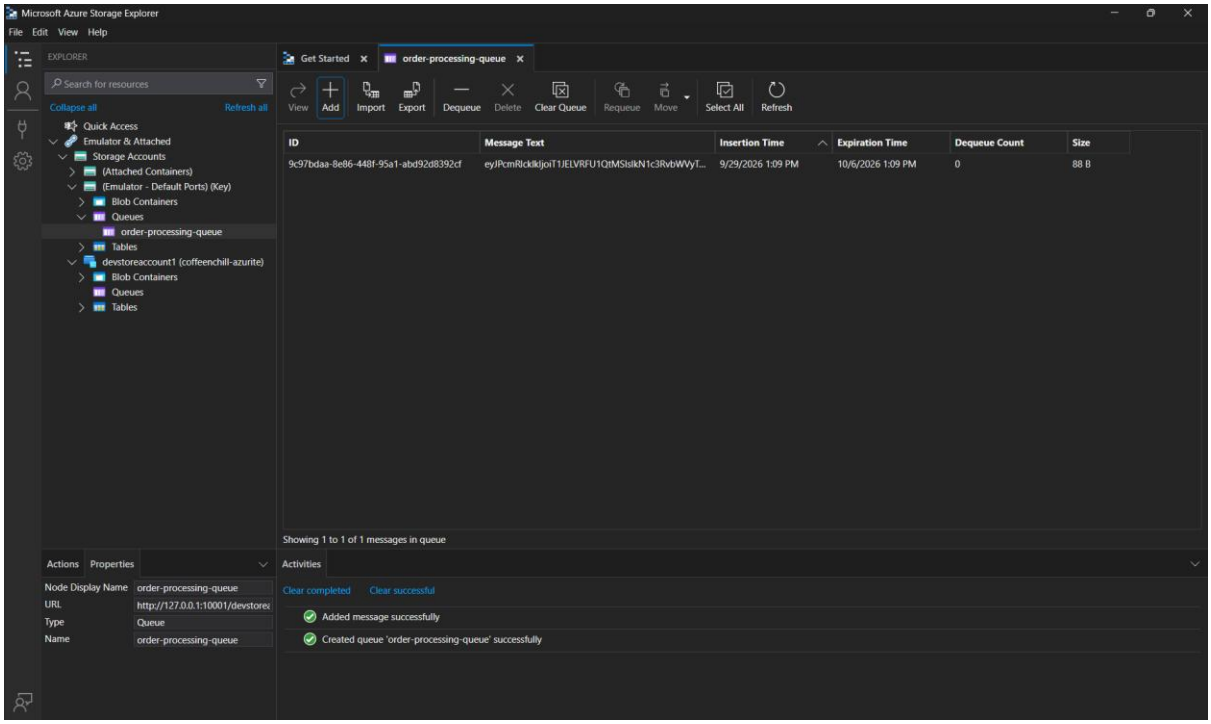

## Queue table after submitting and order (QueueTableaftersubmitorder.png)


Microsoft Azure Storage Explorer
File Edit View Help
EXPLORER
Search for resources
Collapse all
Refresh all
Quick Access
Emulator & Attached
Storage Accounts
(Attached Containers)
(Emulator - Default Ports) (Key)
Blob Containers
Queues
order-processing-queue
student-query-queue
testqueue
Tables
Get Started
order-processing-queue
View
Add
Import
Export
Dequeue
Delete
Clear Queue
Requeue
Move
Select All
Refresh
ID
Message Text
Insertion Time
Expiration Time
Dequeue Count
Size
ad9093c8-899d-4df9-8041-8116e66e305e
{"OrderId":"ORD-2026-8802","CustomerName":"Joh...
9/30/2026 2:48 PM
10/7/2026 2:48 PM
0
204 B
Showing 1 to 1 of 1 messages in queue
Actions
Properties
Node Display Name
order-processing-queue
URL
http://127.0.0.1:10001/devstorea
Type
Queue
Name
order-processing-queue
Activities
Clear completed
Clear successful
ENG
US
3:00 PM
9/30/2026

## Queue Trigger and Order Tracking (Kristen - ST10472683)

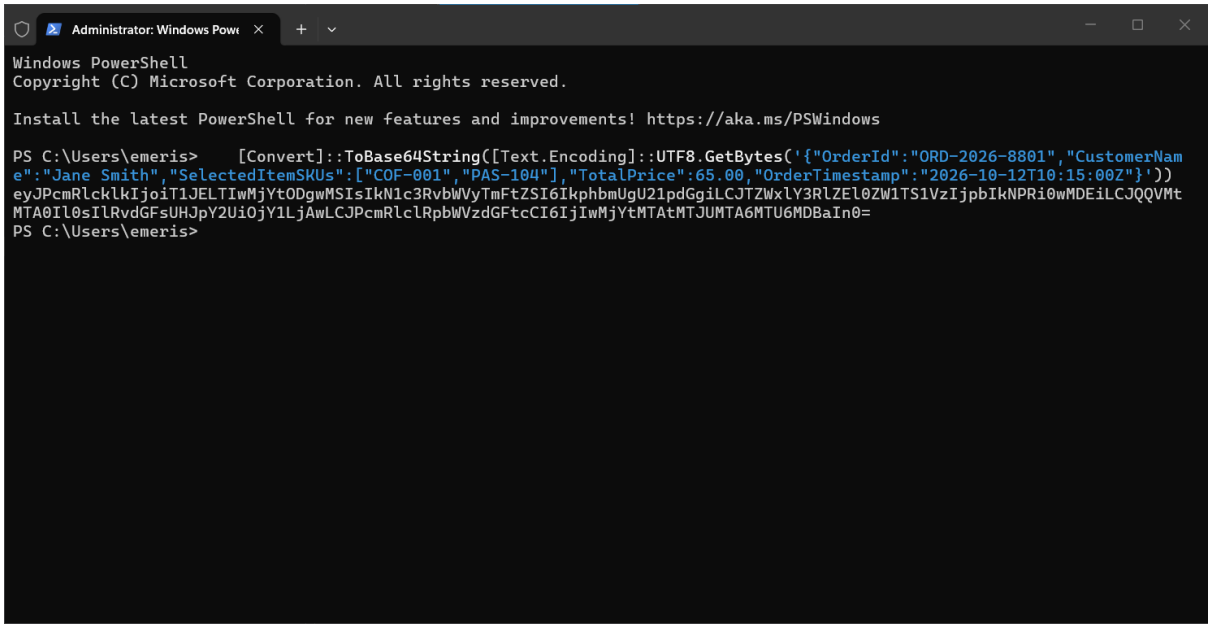
If you want to see each day's progress check \docs\screenshots_part2\kristen.

**QueueTrigger fires: (day 1/ P2_04_trigger_skeleton_log.png)**

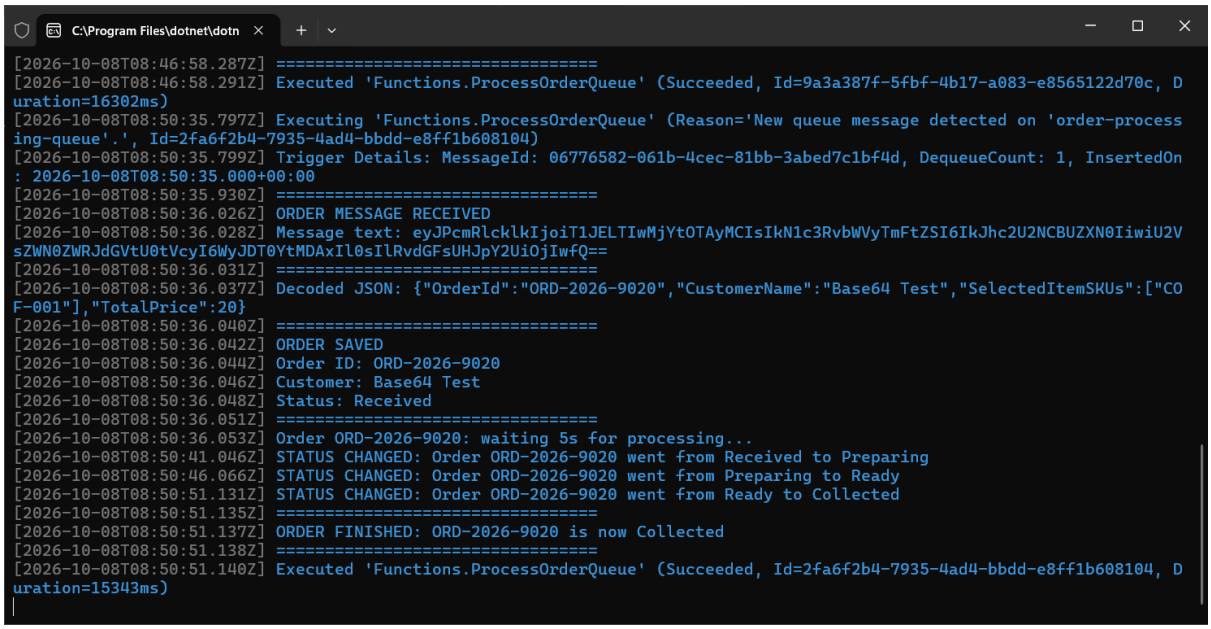



QueueTrigger working by itself.

**Base64 and decoding (day2_encoded_message.png and day8_P2_DecodedJSON_message.png):**

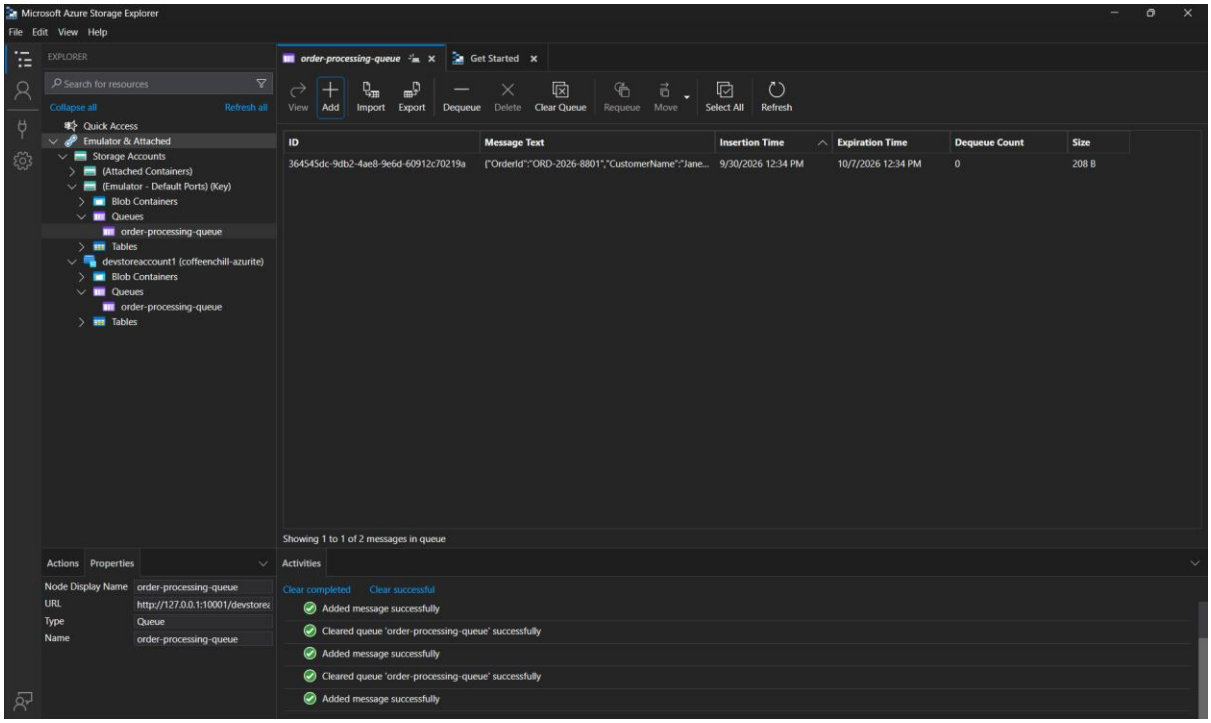


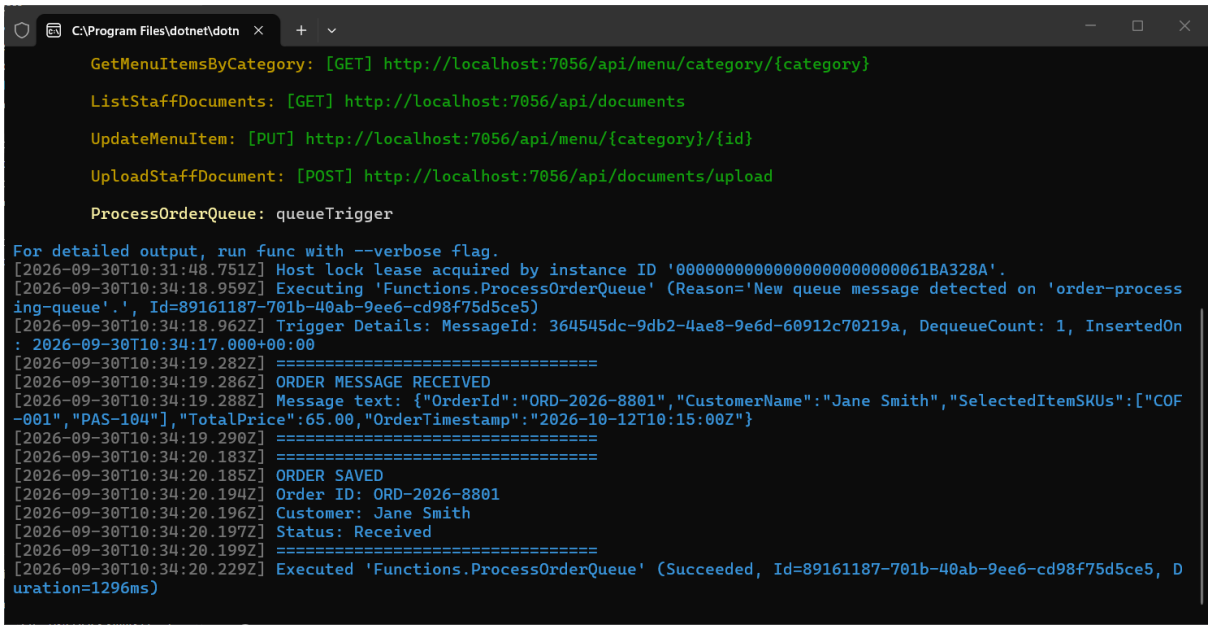

The encoded message




Proof the “Decoded JSON:” line works.

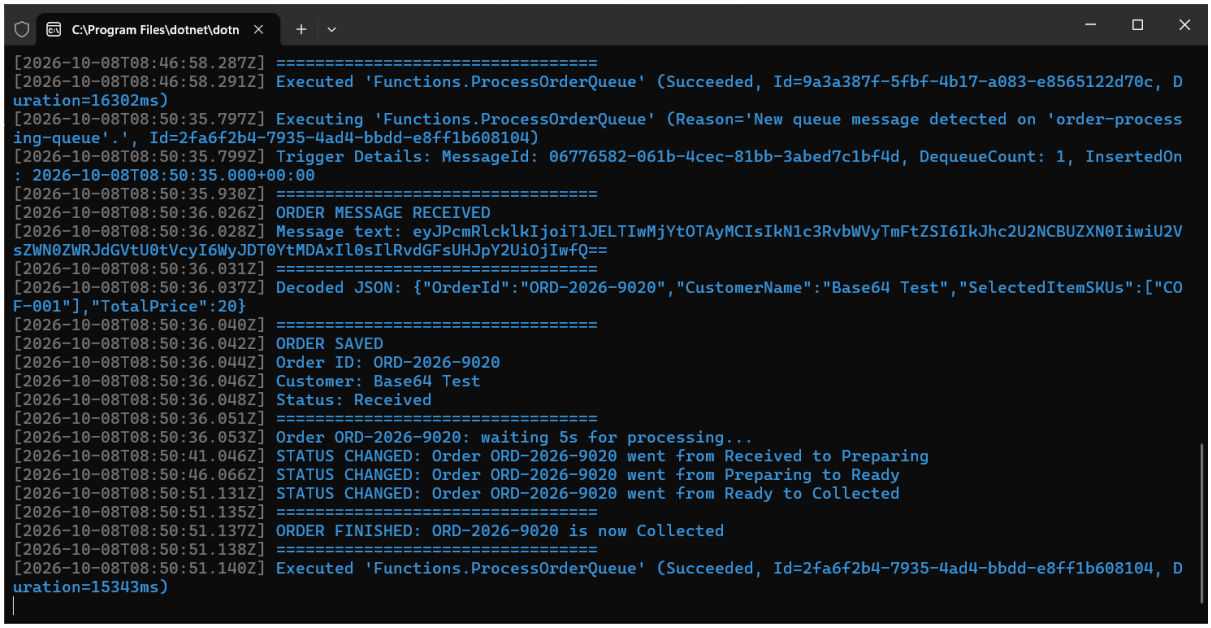
**Order saved: (day2_P2_05_order_saved_log.png and day2_P2_06_orders_table_received.png)(day 2’s order saved)**



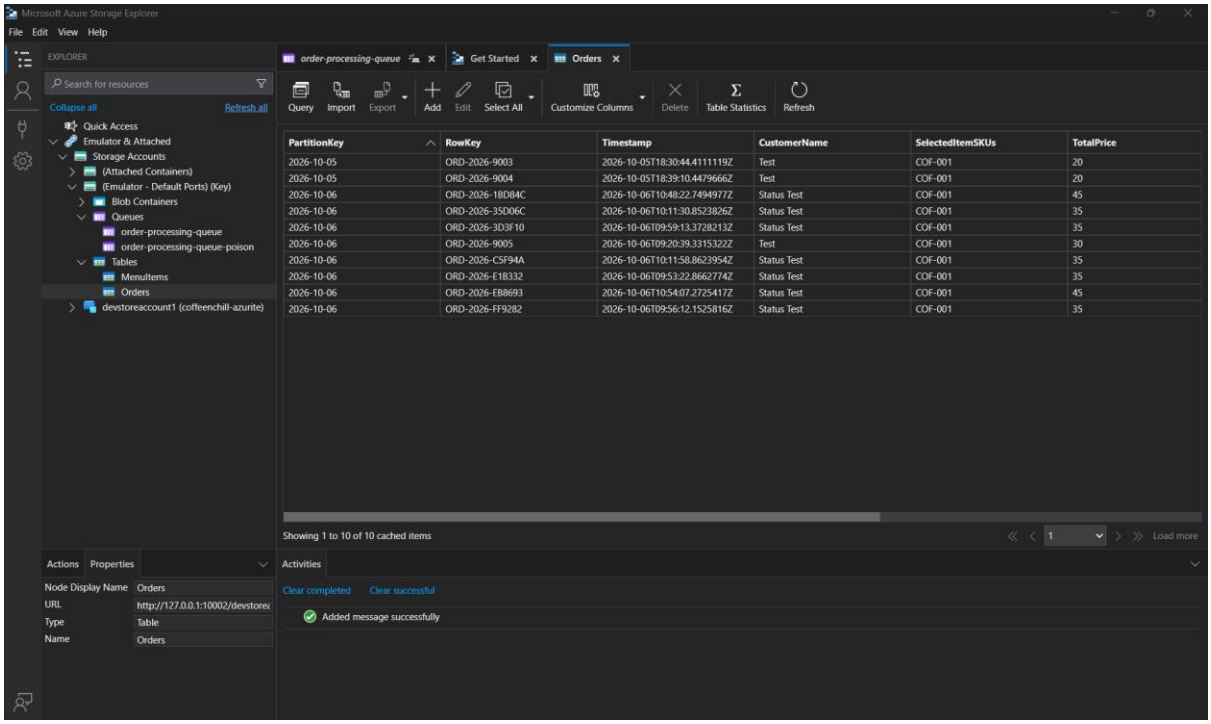



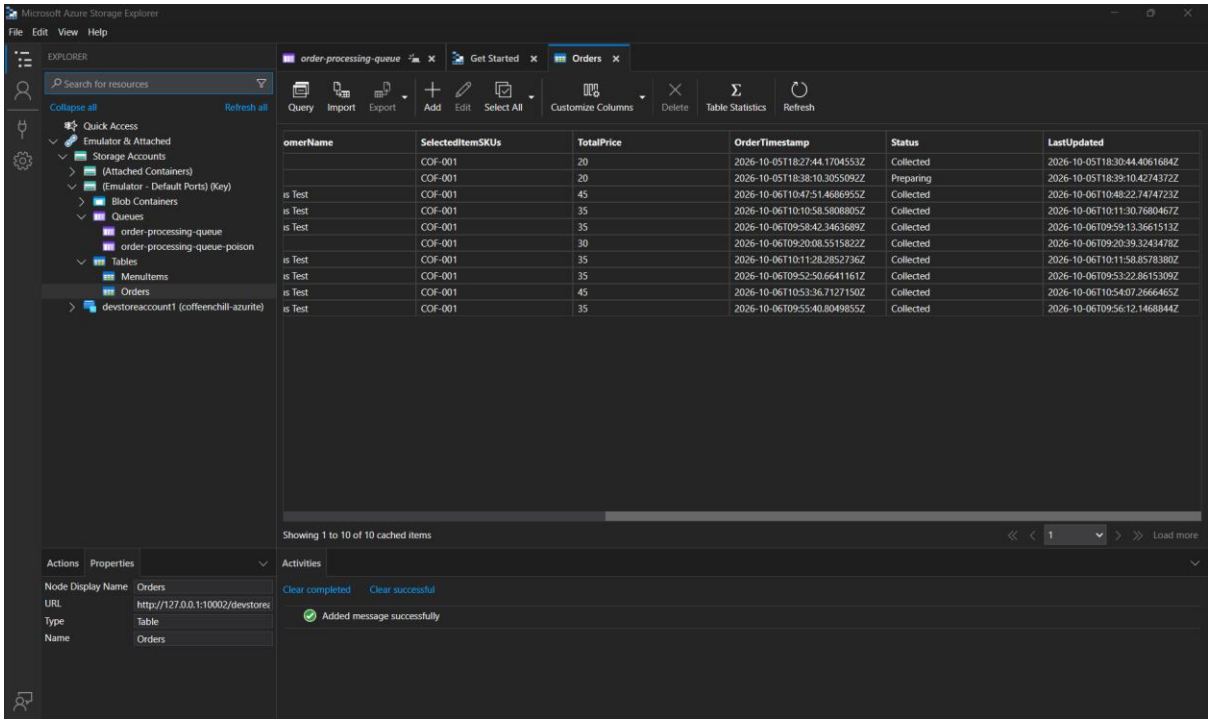

Log and Orders Table with received.

**Status changes: (day8_P2_DecodedJSON_message.png):**

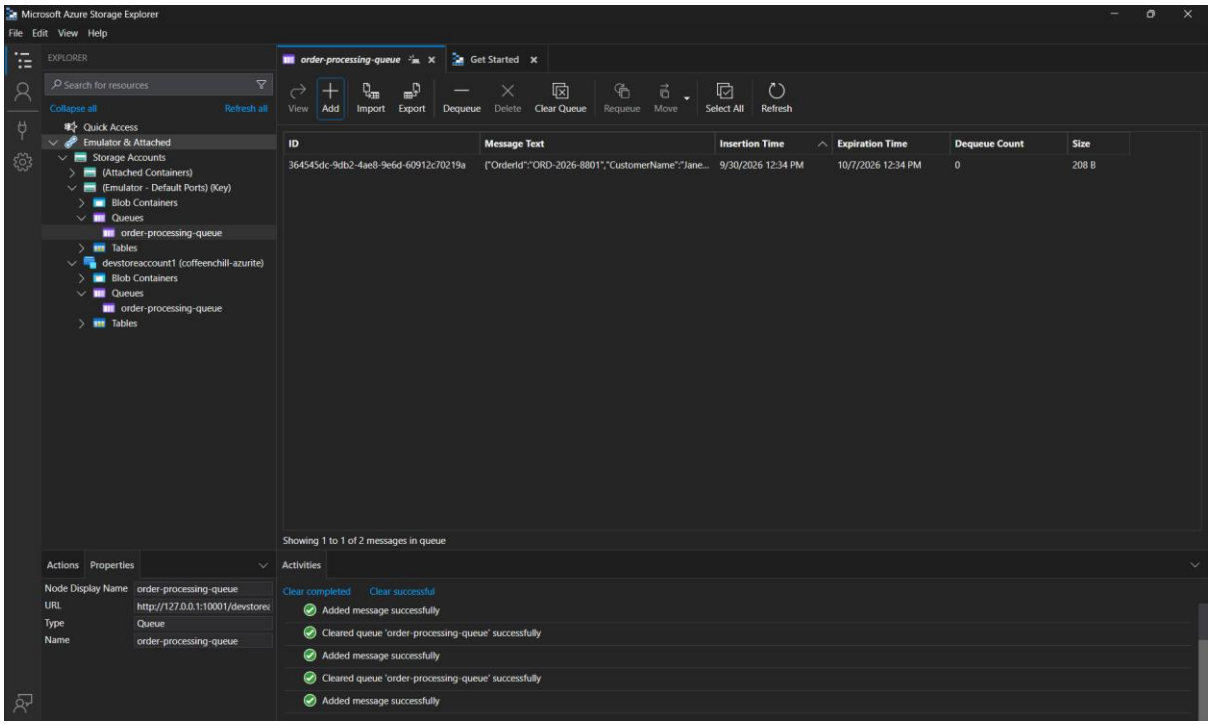



**Final status: (day6_P2_orders_table_collected_1.png and day6_P2_orders_table_collected_2.png)**

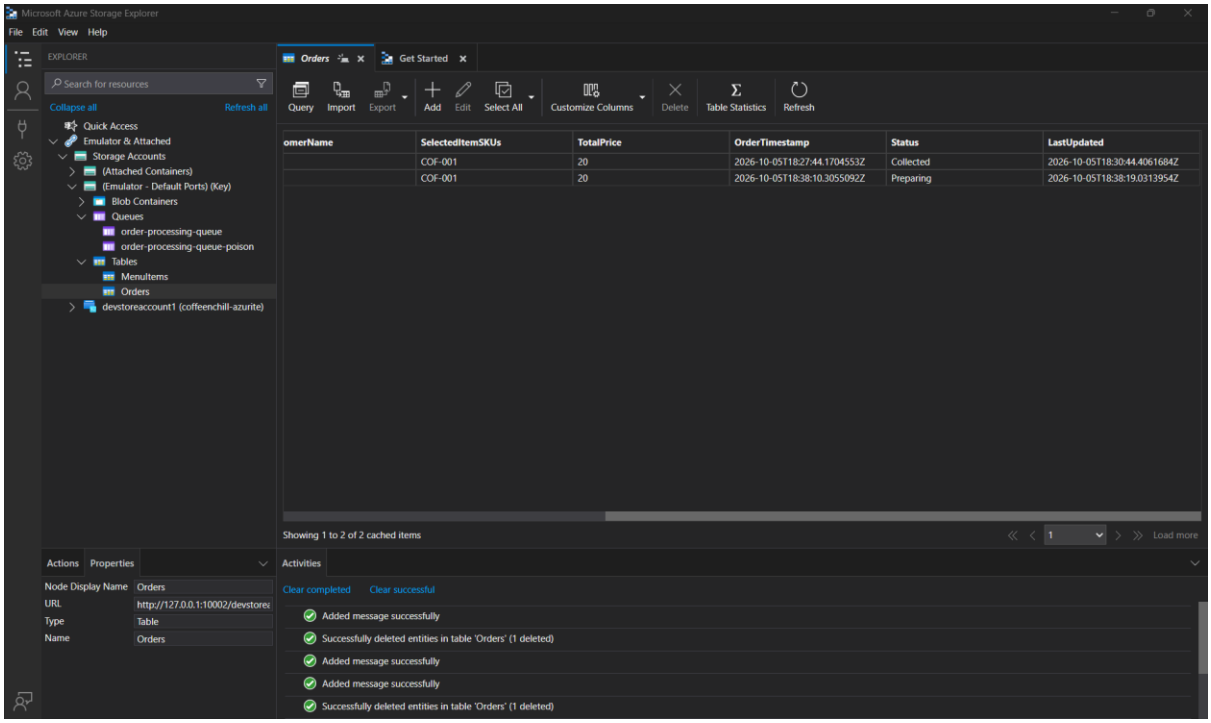




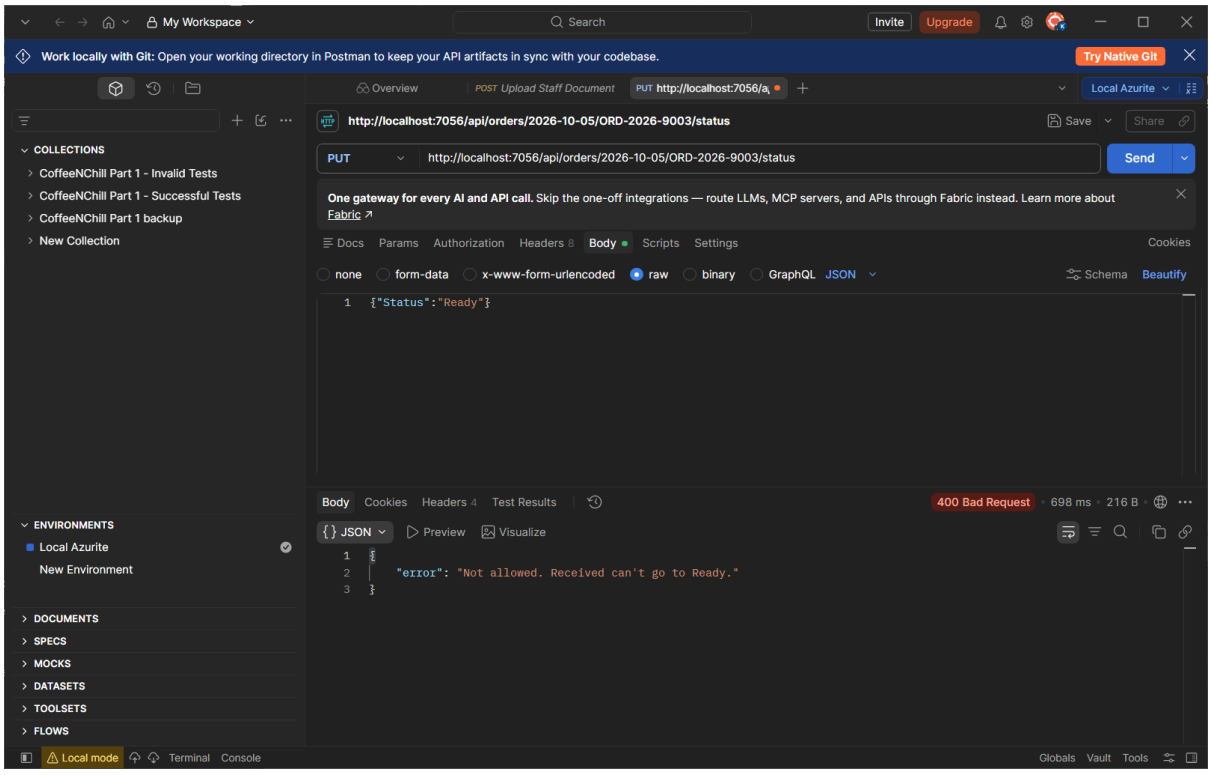

Final version we use from what it used to look like: (day2_P2_05_order_saved_log)



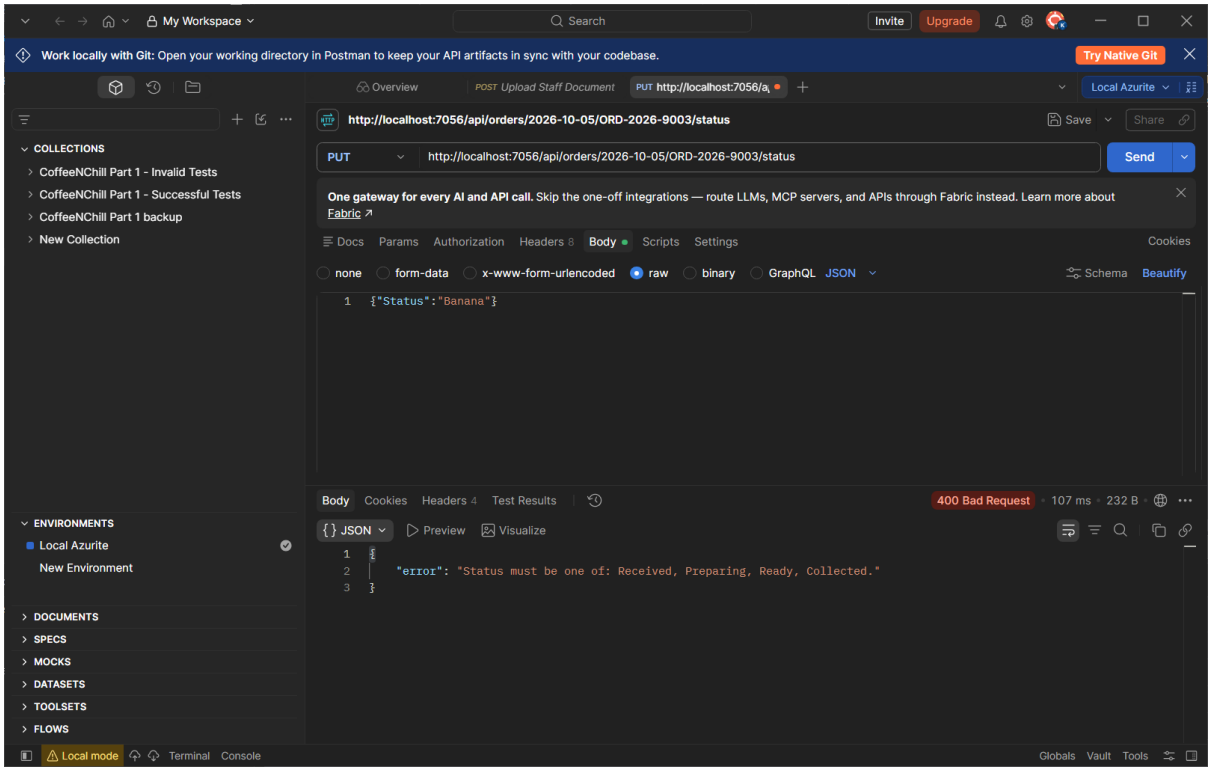

**Status endpoint: (day4_P2_05_put_status_200_updated_status.png, day4_P2_07_put_skip_status_400.png, day4_P2_08_put_invalid_status_400.png and day4_P2_06_put_order_not_found_404.png)**



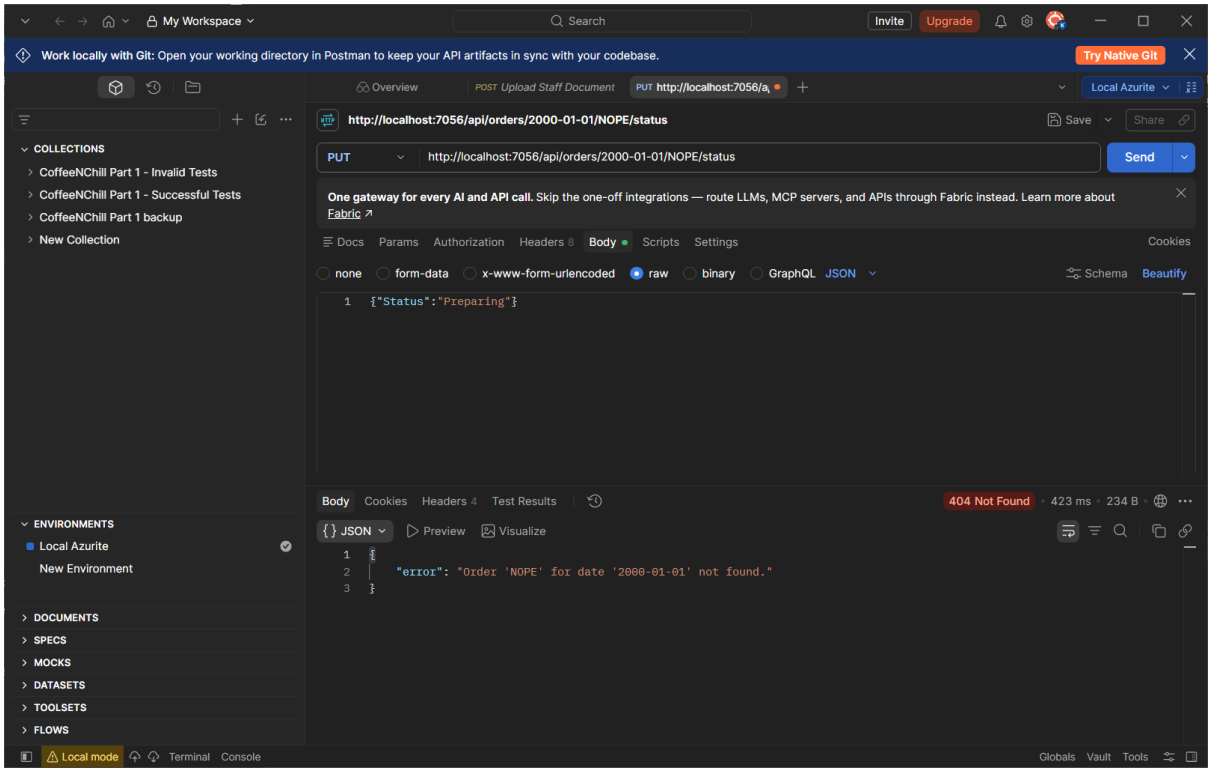

200 results




400 skip status

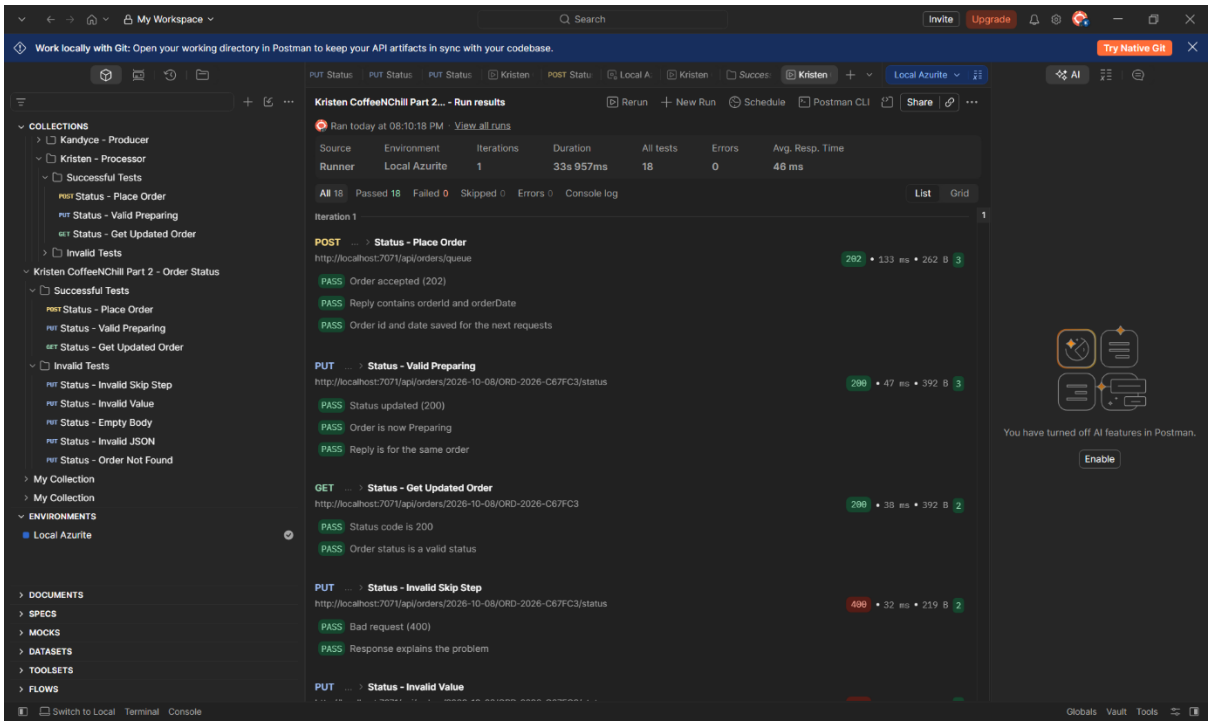


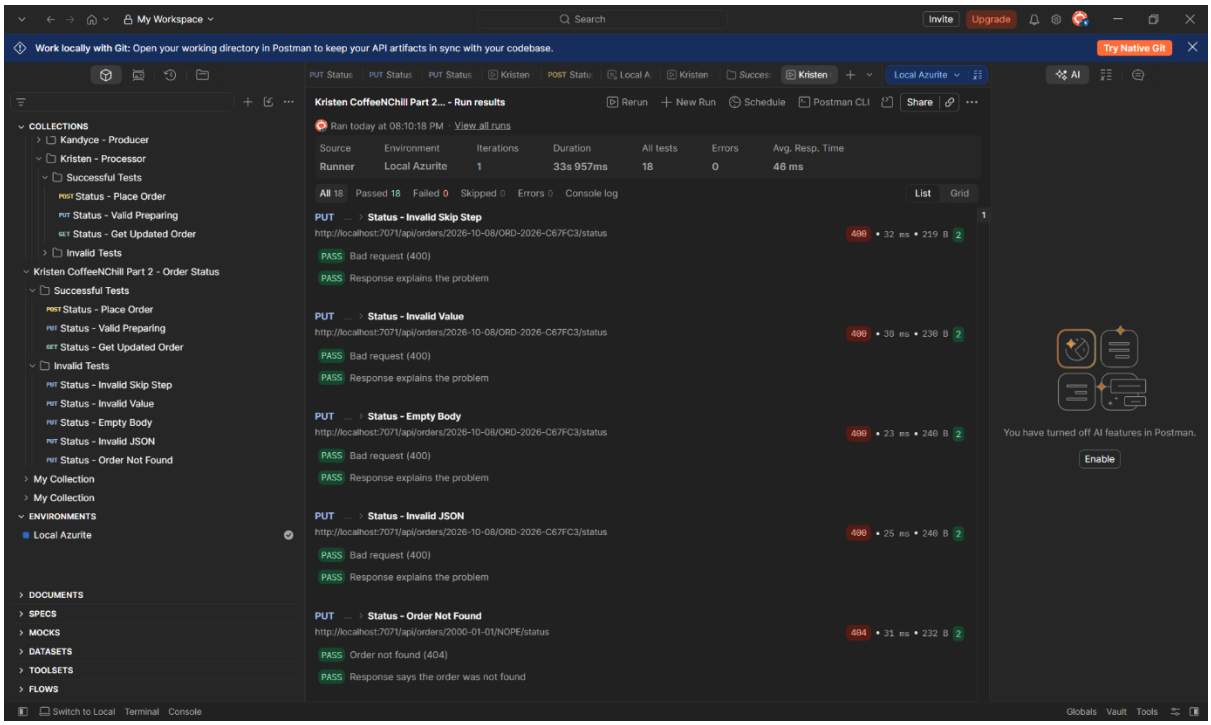

400 invalid status




404 order not found

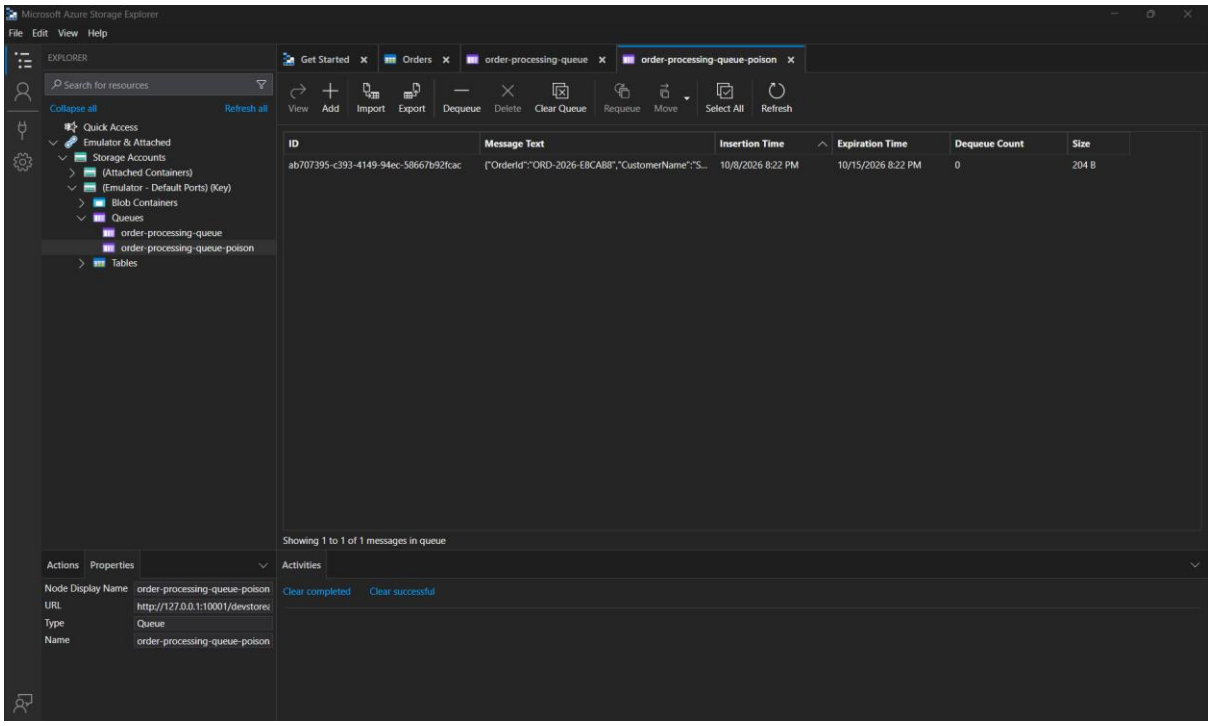
**Postman collection: (day8_P2_successful_postman_run_in_docker1.png and day8_P2_successful_postman_run_in_docker2.png)**



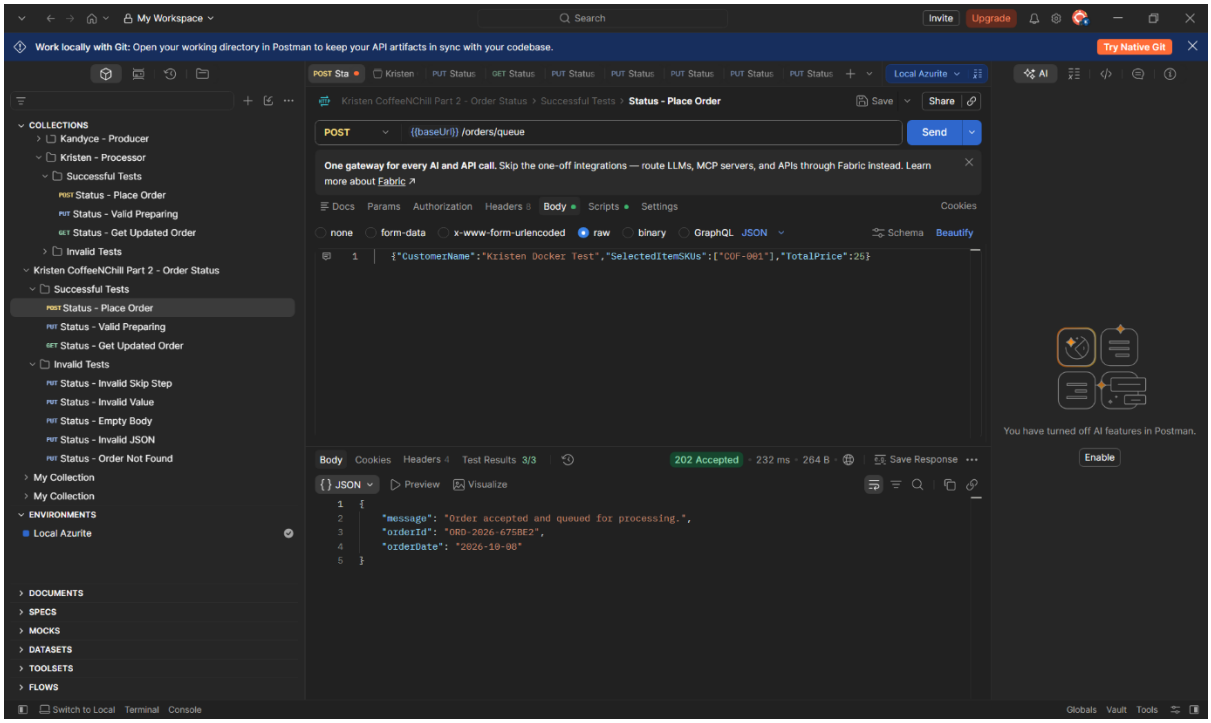



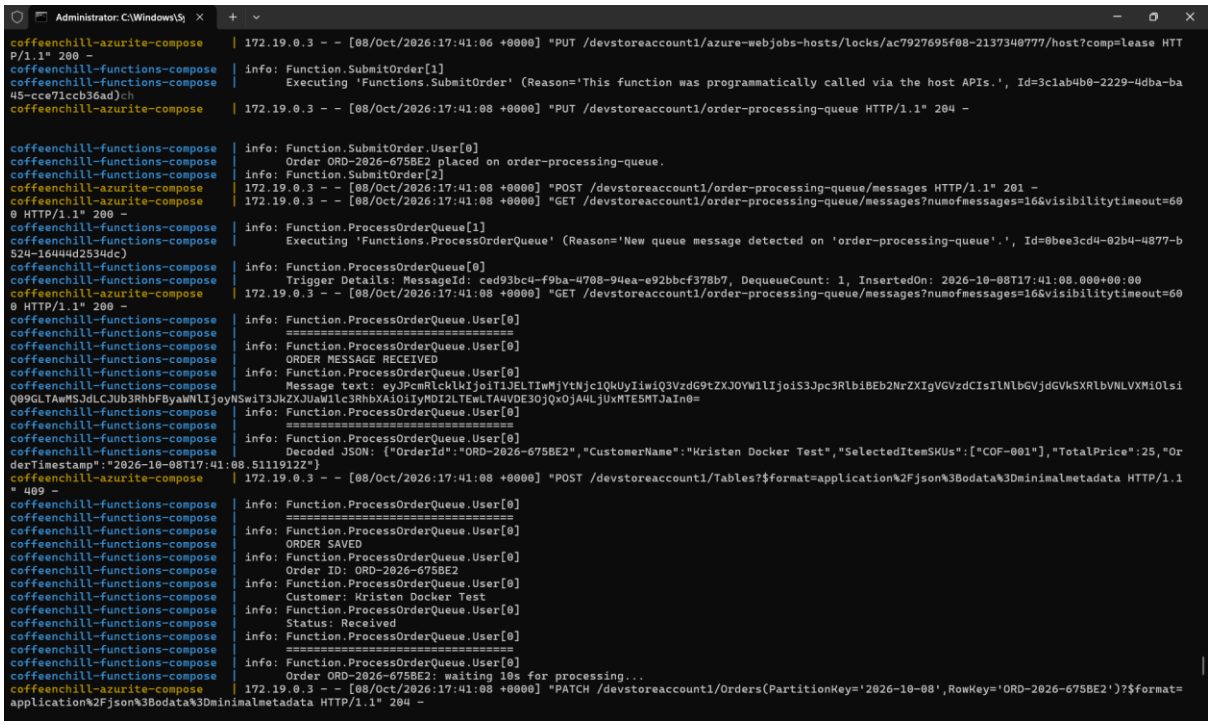

**Poison queue: (day8_P2_successful_poison_testing_in_docker1.png and day8_P2_successful_poison_testing_in_docker2.png )**

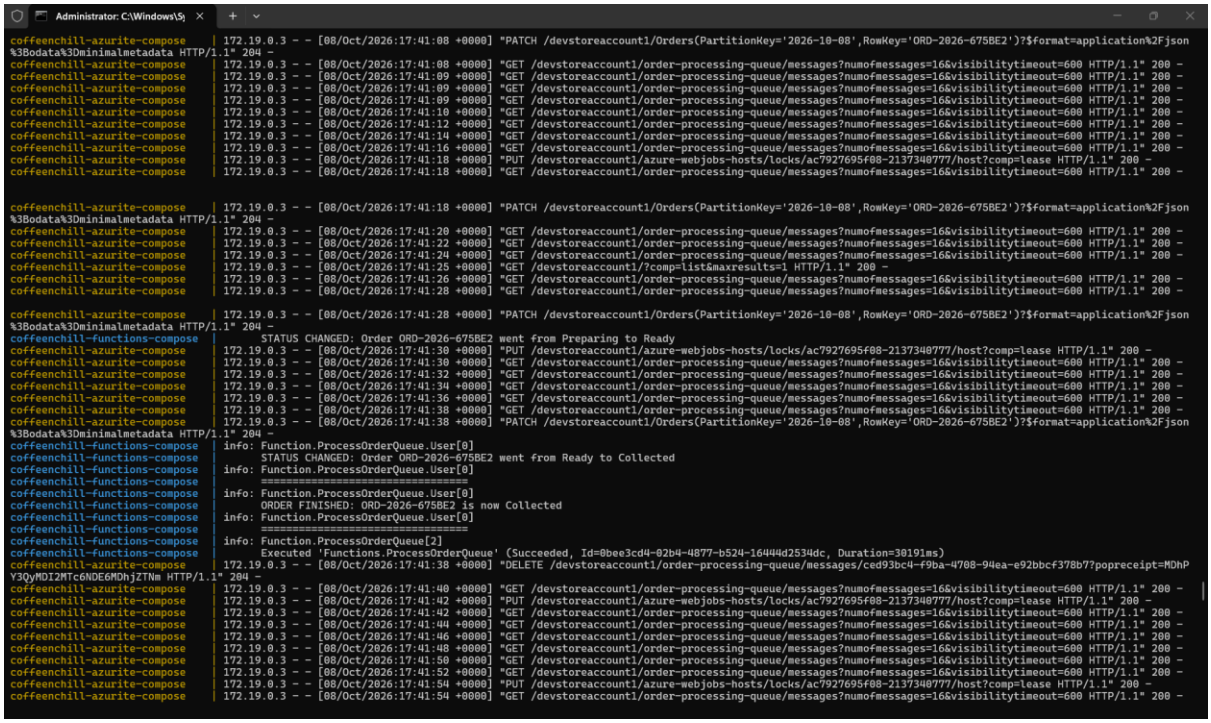



**Docker proof: (day8_P2_Successful_Docker_image_test_PlaceOrder1.png, day8_P2_Successful_Docker_image_test_PlaceOrder2.png and day8_P2_Successful_Docker_image_test_PlaceOrder3.png)**

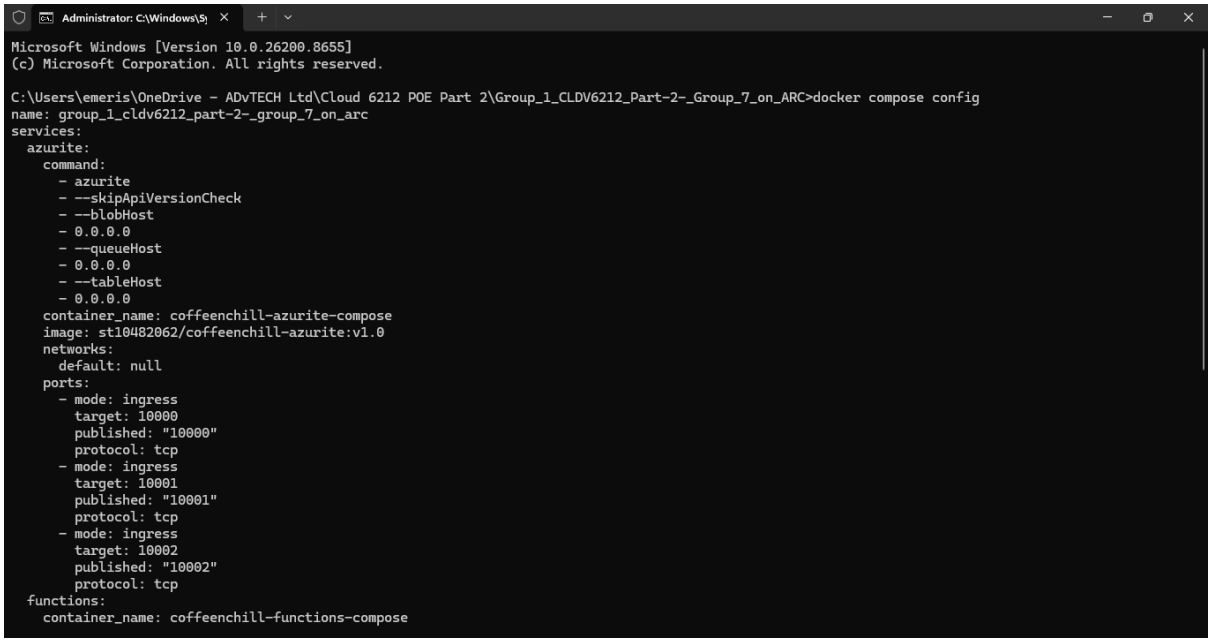

My Workspace
Search
Invite
Upgrade
Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.
Try Native Git
COLLECTIONS
Kandyce - Producer
Kristen - Processor
Successful Tests
Status - Place Order
Status - Valid Preparing
Status - Get Updated Order
Invalid Tests
Kristen CoffeeNChill Part 2 - Order Status
Successful Tests
Status - Place Order
Status - Valid Preparing
Status - Get Updated Order
Invalid Tests
Status - Invalid Skip Step
Status - Invalid Value
Status - Empty Body
Status - Invalid JSON
Status - Order Not Found
My Collection
My Collection
ENVIRONMENTS
Local Azurite
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
Kristen CoffeeNChill Part 2 - Order Status › Successful Tests › Status - Place Order
Save
Share
POST
{{baseUrl}} /orders/queue
Send
One gateway for every AI and API call. Skip the one-off integrations — route LLMs, MCP servers, and APIs through Fabric instead. Learn more about Fabric
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{"CustomerName":"Kristen Docker Test","SelectedItemSKUs":["COF-001"],"TotalPrice":25}
Body
Cookies
Headers 4
Test Results 3/3
202 Accepted
232 ms
264 B
Save Response
JSON
Preview
Visualize
{
"message": "Order accepted and queued for processing.",
"orderId": "ORD-2026-675BE2",
"orderDate": "2026-10-08"
}
You have turned off AI features in Postman.
Enable
Switch to Local
Terminal
Console
Globals
Vault
Tools


Administrator: C:\Windows\S
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:06 +0000] "PUT /devstoreaccount1/azure-webjobs-hosts/locks/ac7927695f08-21373407777/host?comp=lease HTTP/1.1" 200 -
coffeenchill-functions-compose | info: Function.SubmitOrder[1]
Executing 'Functions.SubmitOrder' (Reason='This function was programmatically called via the host APIs.', Id=3c1ab4b0-2229-4dba-ba45-cce71ccb36ad)ch
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:08 +0000] "PUT /devstoreaccount1/order-processing-queue HTTP/1.1" 204 -
coffeenchill-functions-compose | info: Function.SubmitOrder.User[0]
Order ORD-2026-675BE2 placed on order-processing-queue.
coffeenchill-functions-compose | info: Function.SubmitOrder[2]
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:08 +0000] "POST /devstoreaccount1/order-processing-queue/messages HTTP/1.1" 201 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:08 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=60 0 HTTP/1.1" 200 -
coffeenchill-functions-compose | info: Function.ProcessOrderQueue[1]
Executing 'Functions.ProcessOrderQueue' (Reason='New queue message detected on 'order-processing-queue'.', Id=0bee3cd4-02b4-4877-b524-16444d2534dc)
coffeenchill-functions-compose | info: Function.ProcessOrderQueue[0]
Trigger Details: MessageId: ced93bc4-f9ba-4708-94ea-e92bbcf378b7, DequeueCount: 1, InsertedOn: 2026-10-08T17:41:08.000+00:00
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:08 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=60 0 HTTP/1.1" 200 -
coffeenchill-functions-compose | info: Function.ProcessOrderQueue.User[0]
==========
coffeenchill-functions-compose | info: Function.ProcessOrderQueue.User[0]
ORDER MESSAGE RECEIVED
coffeenchill-functions-compose | info: Function.ProcessOrderQueue.User[0]
Message text: eyJPcmRlclkIjoiT1JELTIwMjYtNjc1QkUyIiwiQ3VzdG9tZXJOYW1lIjoiS3Jpc3RlbiBEb2NrZXIgVGVzdCIsIlNlbGVjdGVkSXRlbVNLVXMiOlsiQ09GLTAwMSJdLCJUb3RhbFByaWNlIjoyNSwiT3JkZXJUaW1lc3RhbXAiOiIyMDI2LTEwLTA4VDE3OjQxOjA4LjUxMTE5MTJaIn0=
coffeenchill-functions-compose | info: Function.ProcessOrderQueue.User[0]
==========
coffeenchill-functions-compose | info: Function.ProcessOrderQueue.User[0]
Decoded JSON: {"OrderId":"ORD-2026-675BE2","CustomerName":"Kristen Docker Test","SelectedItemSKUs":["COF-001"],"TotalPrice":25,"OrderTimestamp":"2026-10-08T17:41:08.5111912Z"}
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:08 +0000] "POST /devstoreaccount1/Tables?$format=application%2Fjson%3Bodata%3Dminimalmetadata HTTP/1.1" 409 -
coffeenchill-functions-compose | info: Function.ProcessOrderQueue.User[0]
==========
coffeenchill-functions-compose | info: Function.ProcessOrderQueue.User[0]
ORDER SAVED
coffeenchill-functions-compose | info: Function.ProcessOrderQueue.User[0]
Order ID: ORD-2026-675BE2
coffeenchill-functions-compose | info: Function.ProcessOrderQueue.User[0]
Customer: Kristen Docker Test
coffeenchill-functions-compose | info: Function.ProcessOrderQueue.User[0]
Status: Received
coffeenchill-functions-compose | info: Function.ProcessOrderQueue.User[0]
==========
coffeenchill-functions-compose | info: Function.ProcessOrderQueue.User[0]
Order ORD-2026-675BE2: waiting 10s for processing...
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:08 +0000] "PATCH /devstoreaccount1/Orders(PartitionKey='2026-10-08',RowKey='ORD-2026-675BE2')?$format=application%2Fjson%3Bodata%3Dminimalmetadata HTTP/1.1" 204 -

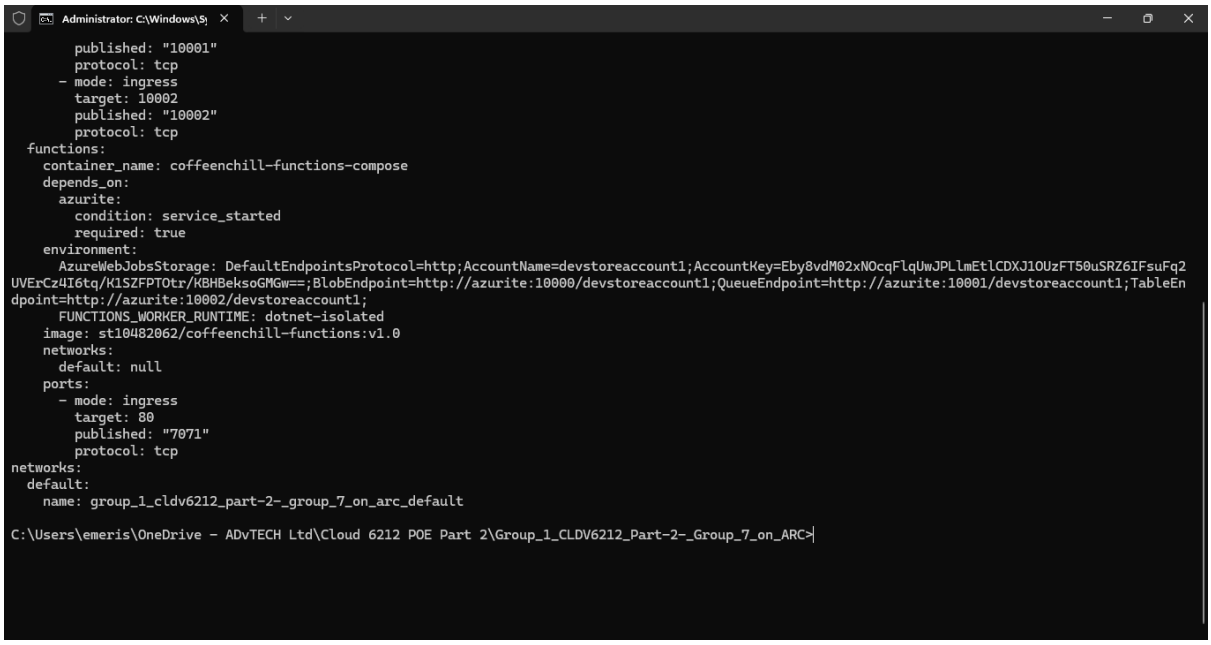

Administrator: C:\Windows\Sy
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:08 +0000] "PATCH /devstoreaccount1/Orders(PartitionKey='2026-10-08',RowKey='ORD-2026-675BE2')?$format=application%2Fjson%3Bodata%3Dminimalmetadata HTTP/1.1" 204 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:08 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:09 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:09 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:09 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:10 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:12 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:14 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:16 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:18 +0000] "PUT /devstoreaccount1/azure-webjobs-hosts/locks/ac7927695f08-2137340777/host?comp=lease HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:18 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:18 +0000] "PATCH /devstoreaccount1/Orders(PartitionKey='2026-10-08',RowKey='ORD-2026-675BE2')?$format=application%2Fjson%3Bodata%3Dminimalmetadata HTTP/1.1" 204 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:20 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:22 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:24 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:25 +0000] "GET /devstoreaccount1/?comp=list&maxresults=1 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:26 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:28 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:28 +0000] "PATCH /devstoreaccount1/Orders(PartitionKey='2026-10-08',RowKey='ORD-2026-675BE2')?$format=application%2Fjson%3Bodata%3Dminimalmetadata HTTP/1.1" 204 -
coffeenchill-functions-compose | STATUS CHANGED: Order ORD-2026-675BE2 went from Preparing to Ready
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:30 +0000] "PUT /devstoreaccount1/azure-webjobs-hosts/locks/ac7927695f08-2137340777/host?comp=lease HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:30 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:32 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:34 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:36 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:38 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:38 +0000] "PATCH /devstoreaccount1/Orders(PartitionKey='2026-10-08',RowKey='ORD-2026-675BE2')?$format=application%2Fjson%3Bodata%3Dminimalmetadata HTTP/1.1" 204 -
coffeenchill-functions-compose | info: Function.ProcessOrderQueue.User[0]
coffeenchill-functions-compose | STATUS CHANGED: Order ORD-2026-675BE2 went from Ready to Collected
coffeenchill-functions-compose | info: Function.ProcessOrderQueue.User[0]
coffeenchill-functions-compose | ==========
coffeenchill-functions-compose | info: Function.ProcessOrderQueue.User[0]
coffeenchill-functions-compose | ORDER FINISHED: ORD-2026-675BE2 is now Collected
coffeenchill-functions-compose | info: Function.ProcessOrderQueue.User[0]
coffeenchill-functions-compose | ==========
coffeenchill-functions-compose | info: Function.ProcessOrderQueue[2]
coffeenchill-functions-compose | Executed 'Functions.ProcessOrderQueue' (Succeeded, Id=0bee3cd4-02b4-4877-b524-16444d2534dc, Duration=30191ms)
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:38 +0000] "DELETE /devstoreaccount1/order-processing-queue/messages/ced93bc4-f9ba-4708-94ea-e92bbcf378b7?popreceipt=MDhPY3QyMDI2MTc6NDE6MDhjZTNm HTTP/1.1" 204 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:40 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:42 +0000] "PUT /devstoreaccount1/azure-webjobs-hosts/locks/ac7927695f08-2137340777/host?comp=lease HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:42 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:44 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:46 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:48 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:50 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:52 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:54 +0000] "PUT /devstoreaccount1/azure-webjobs-hosts/locks/ac7927695f08-2137340777/host?comp=lease HTTP/1.1" 200 -
coffeenchill-azurite-compose | 172.19.0.3 - - [08/Oct/2026:17:41:54 +0000] "GET /devstoreaccount1/order-processing-queue/messages?numofmessages=16&visibilitytimeout=600 HTTP/1.1" 200 -

## Docker Compose and Docker Hub (Thami)

Docker compose (Docker compose config screenshot 1 .png):

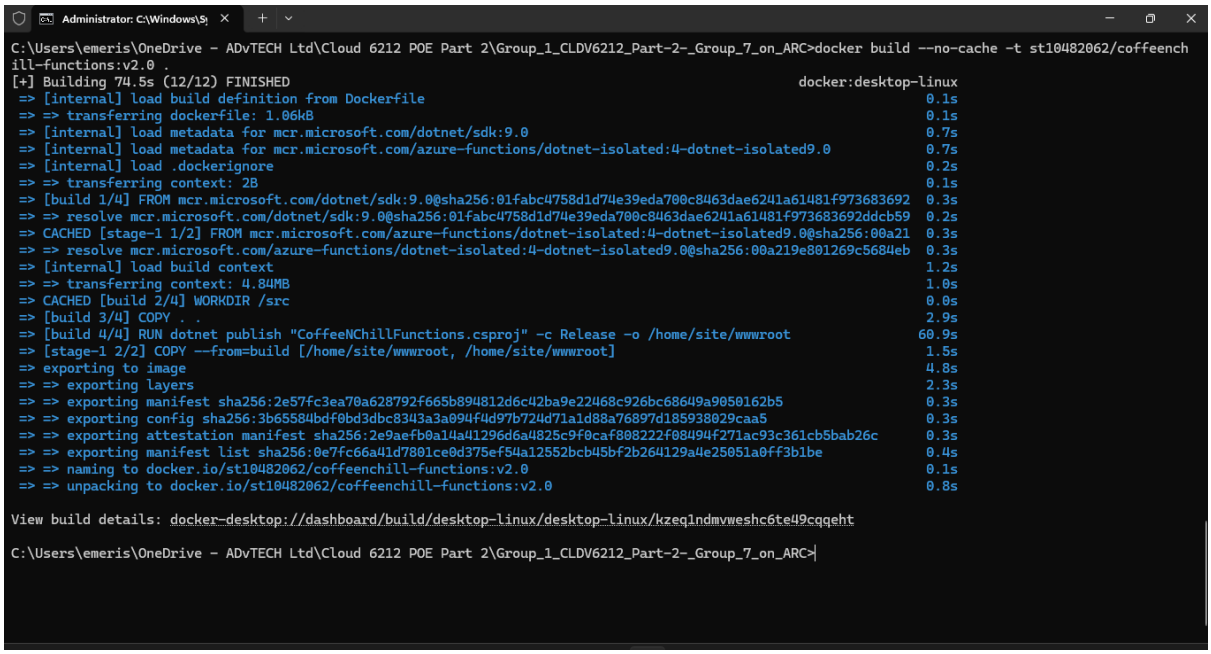



Docker config (Docker compose config screenshot 2 .png):




Docker build and push:

(P2_09_docker_build_final_v2.png):

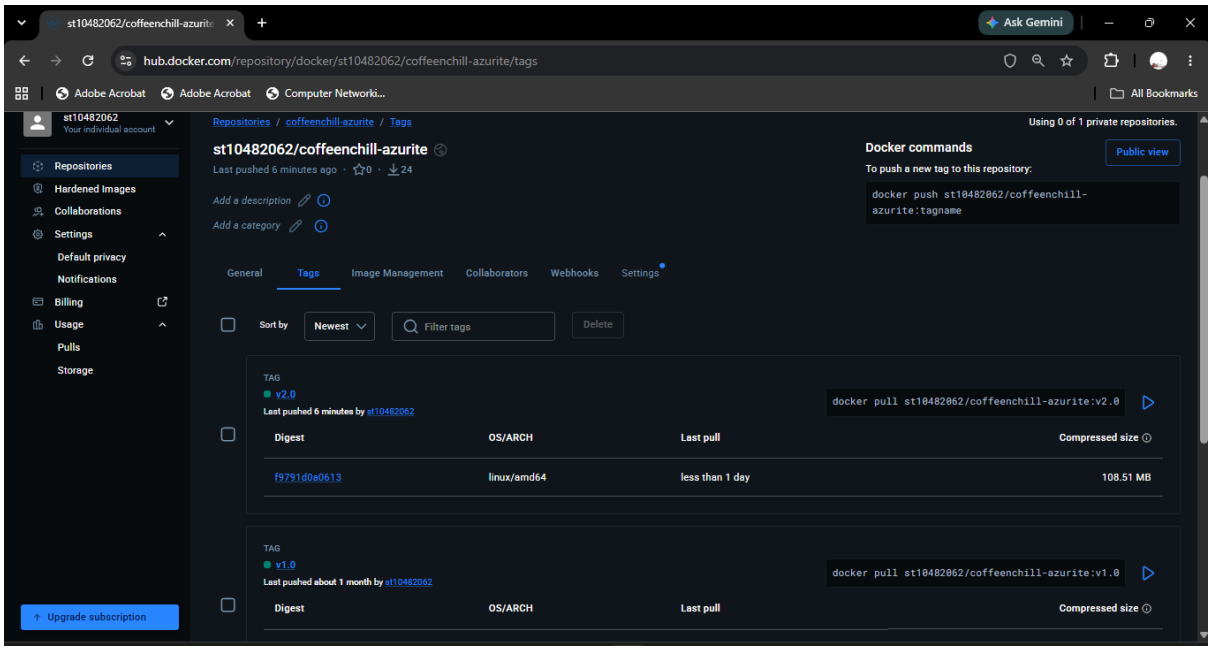



(P2_09_functions_build_push):

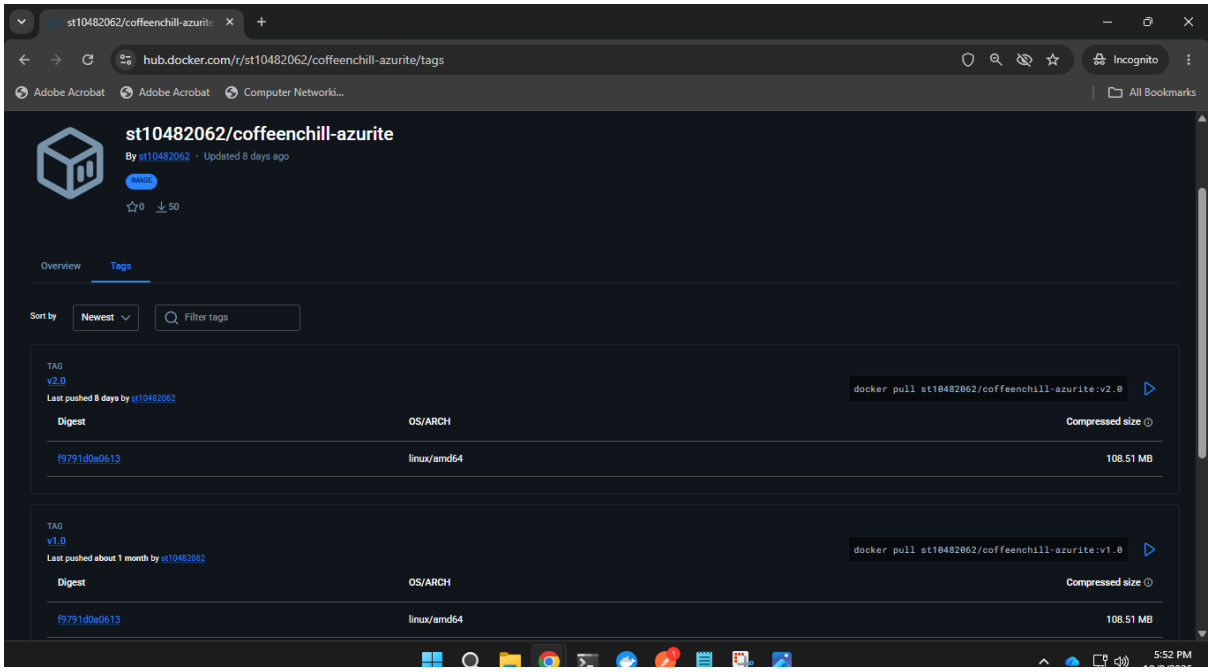



Docker Hub:

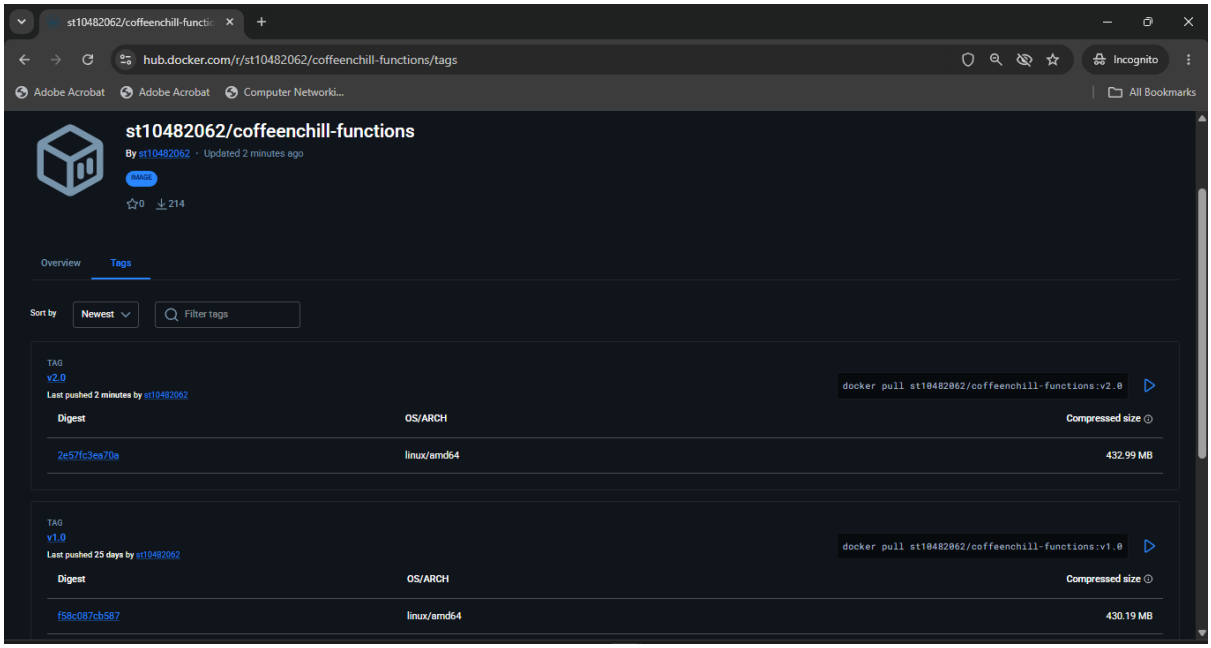
(P2_21_dockerhub_functions_public.png):



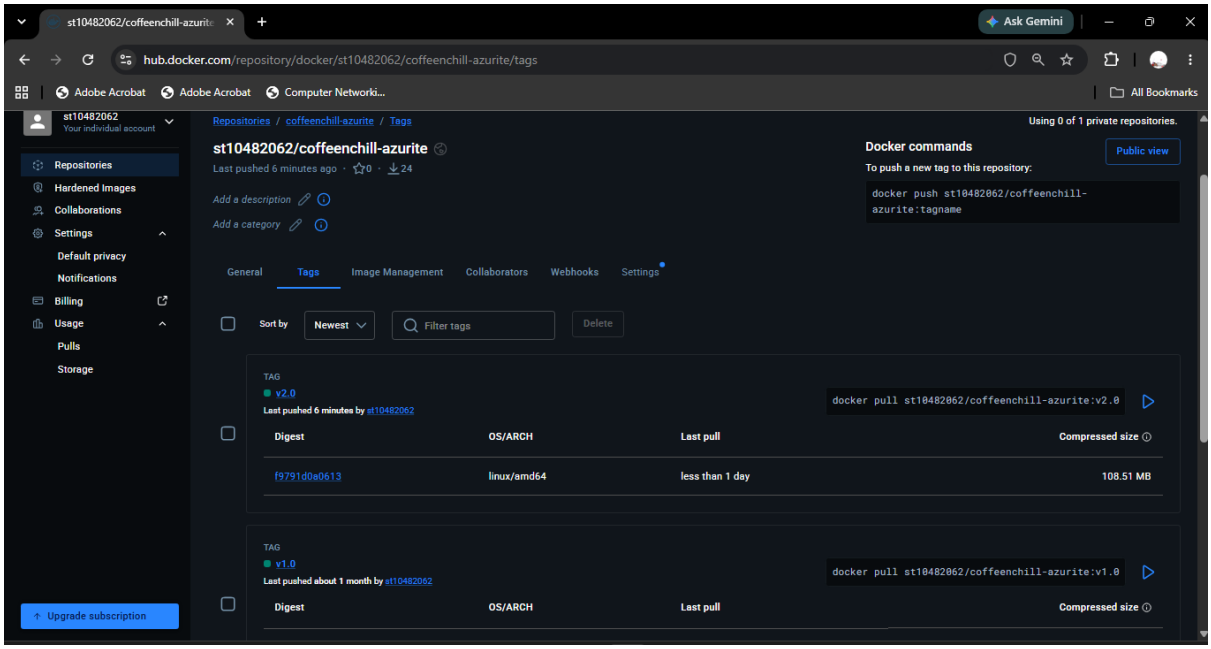

(P2_22_dockerhub_azurite_public.png):




(P2_10_dockerhub_functions_tags.png):




(P2_08_dockerhub_azurite_tags.png):

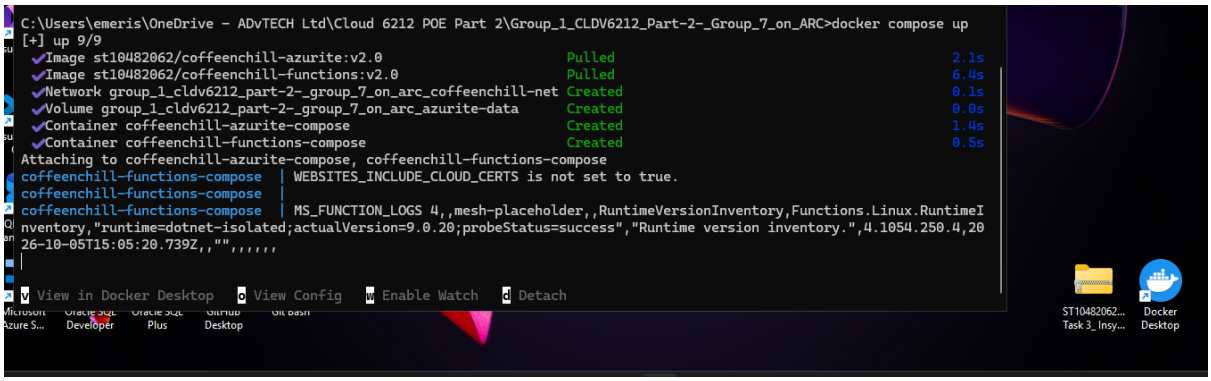



Images pulled from Docker Hub by docker compose up:

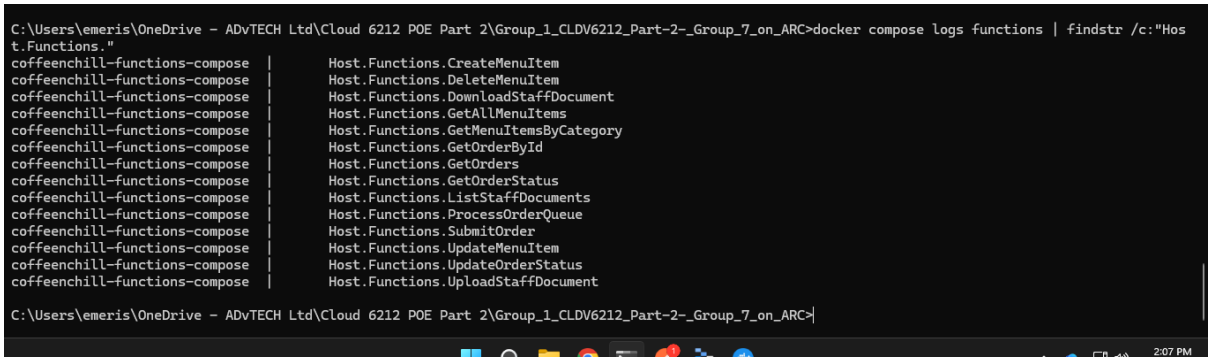
(P2_11_compose_up_pull.png):

```
C:\Users\emeris\OneDrive - ADvTECH Ltd\Cloud 6212 POE Part 2\Group_1_CLDV6212_Part-2-_Group_7_on_ARC>docker compose up
[+] up 8/8
 ✔Image st10482062/coffeenchill-azurite:v2.0                    Pulled                                  2.0s
 ✔Image st10482062/coffeenchill-functions:v2.0                  Pulled                                  5.8s
 ✔Network group_1_cldv6212_part-2-_group_7_on_arc_coffeenchill-net Created                              0.2s
 ✔Container coffeenchill-azurite-compose                        Created                                 1.1s
 ✔Container coffeenchill-functions-compose                      Created                                 0.4s
Attaching to coffeenchill-azurite-compose, coffeenchill-functions-compose
```

(P2_13_compose_up_pulling_v2.png):




CoffeNChill containers (P2_16_container_function_list.png):



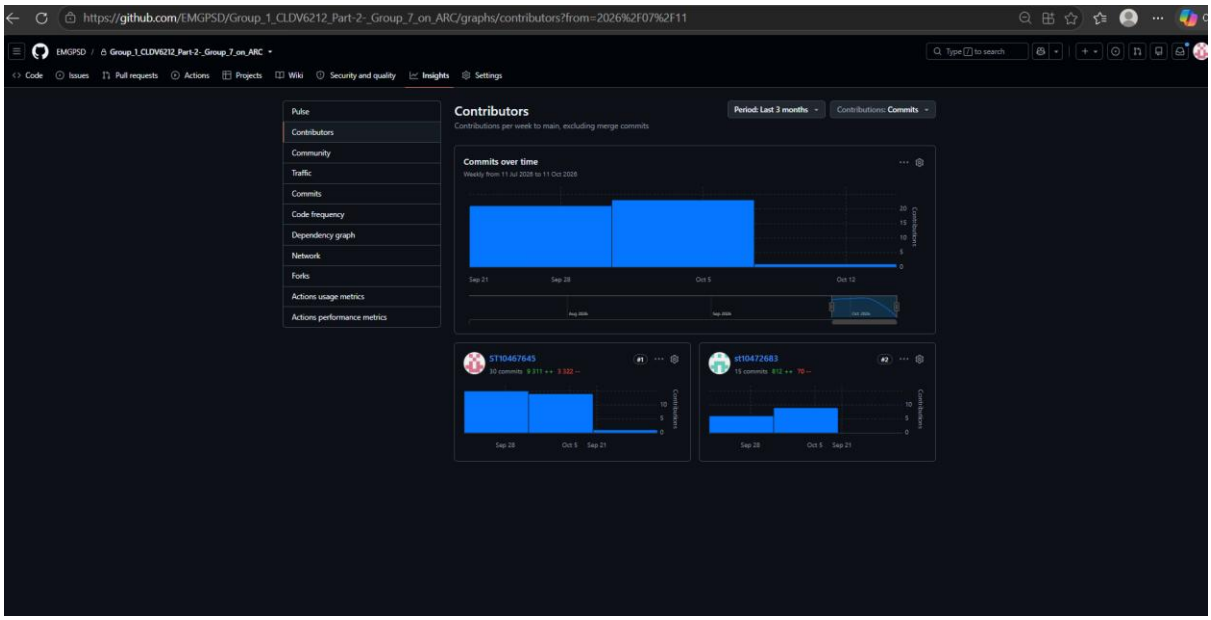
# GitHub Commit History / Contributors

Each member worked on their own branch, these branches were kandyce-part2-producer, kristen-part2-processor and thami-part2-compose, and merging of the branches was done using pull request in an order agreed upon to avoid conflicts during merging. Committing is spread across several days and has descriptive messages like feat:, fix:, docs: and test:. Terminal Output is used to determine the number of commits because GitHub contributors page does not show commits that do not have email address registered to a GitHub account.

## Commit counts and commit days per member (terminal):

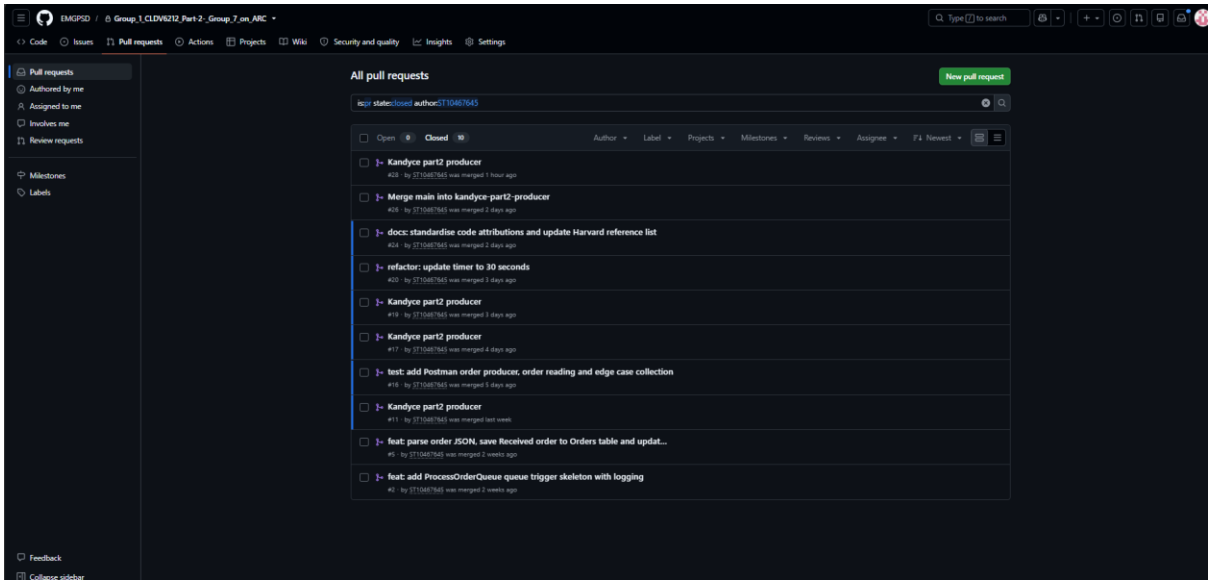
```
C:\Users\kandy\OneDrive - ADvTECH Ltd\source VM Sem 4\repos\CoffeeNChillFunctions Part 2>git shortlog -sn --all --no-merges
    30  ST10467645
    14  ST10472683
    11  ST10482062
```

## GitHub Contributors page:

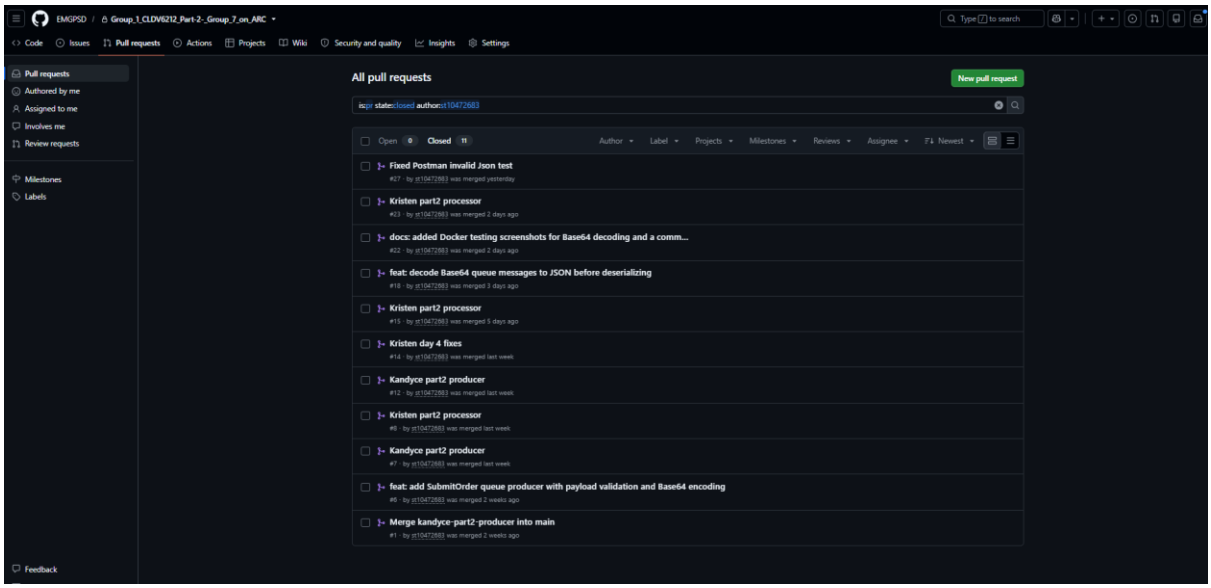



## GitHub Pull Requests Merged:

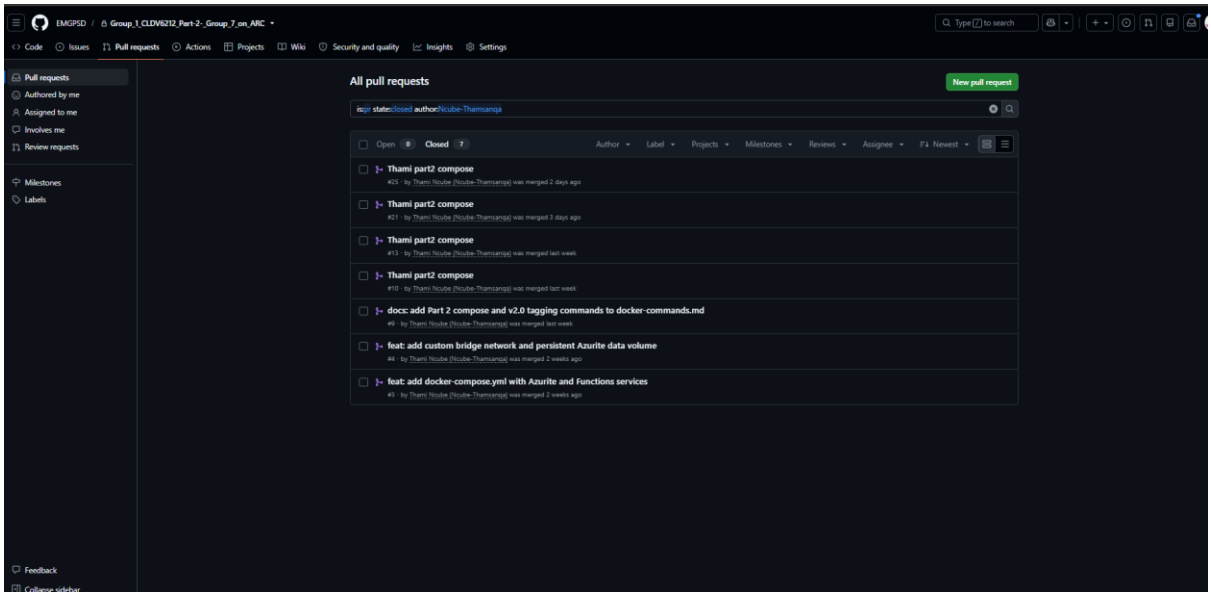
## Kandyce (ST10467645):




## Kristen (ST10472683)




## Thami (ST10482062)


EMGPSD / Group_1_CLDV6212_Part-2-_Group_7_on_ARC
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
Pull requests
Authored by me
Assigned to me
Involves me
Review requests
Milestones
Labels
Feedback
All pull requests
New pull request
is:pr state:closed author:Ncube-Thamsanqa
Open 0
Closed 7
Author
Label
Projects
Milestones
Reviews
Assignee
Newest
Thami part2 compose
#25 · by Thami Ncube (Ncube-Thamsanqa) was merged 2 days ago
Thami part2 compose
#21 · by Thami Ncube (Ncube-Thamsanqa) was merged 3 days ago
Thami part2 compose
#13 · by Thami Ncube (Ncube-Thamsanqa) was merged last week
Thami part2 compose
#10 · by Thami Ncube (Ncube-Thamsanqa) was merged last week
docs: add Part 2 compose and v2.0 tagging commands to docker-commands.md
#9 · by Thami Ncube (Ncube-Thamsanqa) was merged last week
feat: add custom bridge network and persistent Azurite data volume
#4 · by Thami Ncube (Ncube-Thamsanqa) was merged 2 weeks ago
feat: add docker-compose.yml with Azurite and Functions services
#3 · by Thami Ncube (Ncube-Thamsanqa) was merged 2 weeks ago

# TESTING SUMMARY

| Test | Expected | Result |
|---|---|---|
| docker compose up from the Docker Hub images, Visual Studio closed | The stack starts | Pass |
| Valid order | 202 and a row in Orders | Pass |
| Invalid orders (name, items, price) | 400, nothing queued | Pass |
| Order life cycle | Received, Preparing, Ready, Collected, 30 seconds apart | Pass |
| Status endpoint and GET by id | Current status returned | Pass |
| PUT status | 200 for the next step, 400 for a skipped or wrong status, 404 for an unknown order | Pass |
| Poison queue | Bad message appears in order-processing-queue-poison after the retries | Pass |
| Data persistence | Rows survive docker compose down (without -v) | Pass |

# DISCLOSURE

Claude AI (Anthropic) was used to create the three presenter title cards included in the video. The video intro and outro pictures were created using ChatGPT (OpenAI). All the code is checked, tested, modified if necessary, and is fully comprehended by the team member who committed it.

## Image credits

| Image (file) | Where it is used in the video | Generated with |
|---|---|---|
| Part2_Intro_Image.png | Start of the video | ChatGPT (OpenAI) + Claude AI (Anthropic) |
| Part2_Card_IntroToKandyce.png | Before Kandyce's producer segment: pink coffee-cup mascot with envelopes for the Base64 queue messages | Claude AI (Anthropic) |
| Part2_Card_IntroToThami.png | Before Thami's Docker segment: blue coffee-cup mascot with a whale carrying containers for the v2.0 Docker Hub images | Claude AI (Anthropic) |
| Part2_Card_IntroToKristen.png | Before Kristen's trigger segment: green coffee-cup mascot with the four status steps, Received to Collected, 30 seconds apart | Claude AI (Anthropic) |
| Outropng.png | End of the video | ChatGPT (OpenAI) |

These images are decorative graphics for the video. There is no code or output involved, while the texts from the cards (name, student number, and role) were cross-checked against the contributions table of the team members. These images are cited here but not in the reference list as they are AI-generated by our group.

# CODE ATTRIBUTION REGISTER

Every source file starts with an attribution comment. This register lists them with the same reference letters as the reference list. Lecturer-provided material is cited as (IIE, 2026).

### `DeleteMenuItem.cs`

Code Attribution: The Azure Table Storage entity handling and deletion pattern was based on Microsoft's official Azure Tables client library documentation.

> Microsoft Learn (2026c). *Azure Tables client library for .NET - version 12.12.0.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/overview/azure/data.tables-readme?view=azure-dotnet [Accessed 04 September 2026].

### `DownloadStaffDocument.cs`

Code Attribution: The approach for downloading a staff document from Azure Blob Storage was based on Microsoft's official documentation.

> Microsoft Learn (2026d). *Download a blob with .NET.* [Online] Available at: https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blob-download [Accessed 04 September 2026].

### `GetAllMenuItems.cs`

Code Attribution: The Azure Table Storage client pattern for retrieving menu item entities was based on Microsoft's official Azure Tables client library documentation.

> Microsoft Learn (2026c). *Azure Tables client library for .NET - version 12.12.0.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/overview/azure/data.tables-readme?view=azure-dotnet [Accessed 04 September 2026].

### `GetMenuItemsByCategory.cs`

Code Attribution: The approach to querying Azure Table Storage for menu items by category was informed by Microsoft's documentation on Azure Table Storage.

> Microsoft Learn (2022). *What is Azure Table storage?* [Online] Available at: https://learn.microsoft.com/en-us/azure/storage/tables/table-storage-overview [Accessed 04 September 2026].

### `CreateMenuItem.cs`

Code Attribution: The HTTP-triggered Azure Function pattern was based on Microsoft's Azure Functions documentation. Azure Table Storage entity handling was based on Microsoft's Azure Tables client library documentation.

> Microsoft Learn (2025b). *Azure Functions HTTP trigger.* [Online] Available at: https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-http-webhook-trigger?tabs=python-v2%2Cisolated-process%2Cnodejsv4%2Cfunctionsv2&pivots=programming-language-csharp [Accessed 04 September 2026].
>
> Microsoft Learn (2026c). *Azure Tables client library for .NET - version 12.12.0.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/overview/azure/data.tables-readme?view=azure-dotnet [Accessed 04 September 2026].

### `UpdateMenuItem.cs`

Code Attribution: The Azure Table Storage client pattern for updating menu item entities was based on Microsoft's official Azure Tables client library documentation.

> Microsoft Learn (2026c). *Azure Tables client library for .NET - version 12.12.0.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/overview/azure/data.tables-readme?view=azure-dotnet [Accessed 04 September 2026].

### GetOrderById.cs

Code Attribution: Azure Table Storage entity handling was based on Microsoft's official Azure Tables client library documentation. The course example informed the general structure of the function (CLDV6212 lecturer-provided course example, IIE, 2026).

Microsoft Learn (2026c). *Azure Tables client library for .NET - version 12.12.0.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/overview/azure/data.tables-readme?view=azure-dotnet [Accessed 04 September 2026].

### GetOrders.cs

Code Attribution: The Azure Table Storage client pattern for retrieving order entities was based on Microsoft's official Azure Tables client library documentation.

Microsoft Learn (2026c). *Azure Tables client library for .NET - version 12.12.0.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/overview/azure/data.tables-readme?view=azure-dotnet [Accessed 04 September 2026].

### GetOrderStatus.cs

Code Attribution: The Azure Table Storage client pattern for retrieving order status information was based on Microsoft's official Azure Tables client library documentation.

Microsoft Learn (2026c). *Azure Tables client library for .NET - version 12.12.0.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/overview/azure/data.tables-readme?view=azure-dotnet [Accessed 04 September 2026].

### ListStaffDocuments.cs

Code Attribution: The use of Azure Blob Storage to store and list staff documents was based on Microsoft's official Azure Blob Storage documentation.

Microsoft Learn (2026e). *Introduction to Azure Blob Storage.* [Online] Available at: https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-introduction [Accessed 04 September 2026].

### UploadStaffDocument.cs

Code Attribution: Multipart form-data reading was based on Microsoft's MultipartReader documentation. The use of Azure Blob Storage for staff document uploads was based on Microsoft's Blob Storage documentation. Blob HTTP headers and upload options were based on the corresponding Azure SDK documentation.

Microsoft Learn (2026f). *MultipartReader Class.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.webutilities.multipartreader?view=aspnetcore-10.0 [Accessed 04 September 2026].

Microsoft Learn (2026e). *Introduction to Azure Blob Storage.* [Online] Available at: https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-introduction [Accessed 04 September 2026].

Microsoft Learn (2026u). *BlobHttpHeaders Class.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/azure.storage.blobs.models.blobhttpheaders?view=azure-dotnet [Accessed 04 September 2026].

Microsoft Learn (2026v). *BlobUploadOptions.HttpHeaders Property.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/azure.storage.blobs.models.blobuploadoptions.httpheaders?view=azure-dotnet [Accessed 04 September 2026].

### SubmitOrder.cs

Code Attribution: The HTTP-triggered Azure Function pattern was based on Microsoft's Azure Functions documentation. Sending order messages to Azure Queue Storage was based on the Azure Queue Storage client library documentation. Base64 encoding of the message was based on the Convert.ToBase64String

documentation. Handling a failed queue connection was based on the RequestFailedException documentation. The queue trigger that reads the message was based on the Azure Functions queue trigger documentation.


Microsoft Learn (2025b). *Azure Functions HTTP trigger.* [Online] Available at: https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-http-webhook-trigger?tabs=python-v2%2Cisolated-process%2Cnodejsv4%2Cfunctionsv2&pivots=programming-language-csharp [Accessed 04 September 2026].

Microsoft Learn (2026q). *Azure Queue Storage client library for .NET.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/overview/azure/storage.queues-readme?view=azure-dotnet [Accessed 04 September 2026].

Microsoft Learn (2026w). *Convert.ToBase64String Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.convert.tobase64string?view=net-10.0 [Accessed 09 October 2026].

Microsoft Learn (2026r). *RequestFailedException Class.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/azure.requestfailedexception?view=azure-dotnet [Accessed 01 October 2026].

Microsoft Learn (2026g). *Azure Queue storage trigger for Azure Functions.* [Online] Available at: https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-storage-queue-trigger [Accessed 29 September 2026].


### `ProcessOrderQueue.cs`

Code Attribution: The queue-triggered Azure Function pattern and poison-message handling were based on Microsoft's Azure Functions queue trigger documentation. The conditional operator, String.Join, Task.Delay, environment variable retrieval, integer parsing, Base64 decoding and text decoding patterns were based on the relevant .NET documentation. Updating order entities was based on the Azure Tables client library documentation.


Microsoft Learn (2026g). *Azure Queue storage trigger for Azure Functions.* [Online] Available at: https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-storage-queue-trigger [Accessed 29 September 2026].

Microsoft Learn (2026h). *Conditional operator.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/conditional-operator [Accessed 30 September 2026].

Microsoft Learn (2026i). *String.Join Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.string.join?view=net-10.0 [Accessed 30 September 2026].

Microsoft Learn (2026j). *Task.Delay Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.threading.tasks.task.delay?view=net-10.0 [Accessed 01 October 2026].

Microsoft Learn (2026k). *TableClient.UpsertEntityAsync Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/azure.data.tables.tableclient.upsertentityasync?view=azure-dotnet [Accessed 01 October 2026].

Microsoft Learn (2026l). *Poison messages in Azure Functions.* [Online] Available at: https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-storage-queue-trigger#poison-messages [Accessed 01 October 2026].

Microsoft Learn (2026m). *Environment.GetEnvironmentVariable Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.environment.getenvironmentvariable?view=net-10.0 [Accessed 01 October 2026].

Microsoft Learn (2026n). *Int32.TryParse Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.int32.tryparse?view=net-10.0 [Accessed 05 October 2026].

Microsoft Learn (2026o). *Convert.FromBase64String Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.convert.frombase64string?view=net-10.0 [Accessed 08 October 2026].

Microsoft Learn (2026p). *Encoding.GetString Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.text.encoding.getstring?view=net-10.0 [Accessed 08 October 2026].

### `UpdateOrderStatus.cs`

Code Attribution: The HTTP-triggered Azure Function pattern was based on Microsoft's Azure Functions documentation. Request failure handling, JSON deserialization and asynchronous request-body reading were based on the corresponding Microsoft .NET and Azure SDK documentation.


Microsoft Learn (2025b). *Azure Functions HTTP trigger.* [Online] Available at: https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-http-webhook-trigger?tabs=python-v2%2Cisolated-process%2Cnodejsv4%2Cfunctionsv2&pivots=programming-language-csharp [Accessed 04 September 2026].

Microsoft Learn (2026r). *RequestFailedException Class.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/azure.requestfailedexception?view=azure-dotnet [Accessed 01 October 2026].

Microsoft Learn (2026s). *JsonSerializer.Deserialize Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.text.json.jsonserializer.deserialize?view=net-10.0 [Accessed 05 October 2026].

Microsoft Learn (2026t). *StreamReader.ReadToEndAsync Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.io.streamreader.readtoendasync?view=net-10.0 [Accessed 05 October 2026].


### `MenuItem.cs`

Code Attribution: The Azure Table Storage entity model pattern was based on Microsoft's official Azure Tables client library documentation.


Microsoft Learn (2026c). *Azure Tables client library for .NET - version 12.12.0.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/overview/azure/data.tables-readme?view=azure-dotnet [Accessed 04 September 2026].


### `Order.cs`

Code Attribution: The order model structure was adapted from the StudentQuery model in the lecturer-provided Student Relations course example (IIE, 2026). The order fields were based on the CLDV6212 POE brief (IIE, 2026).

### `OrderEntity.cs`

Code Attribution: The order table entity model was adapted from the StudentQueryEntity example provided in the lecturer's course materials (IIE, 2026). The ITableEntity implementation follows Microsoft's official Azure Tables client library documentation.


Microsoft Learn (2026c). *Azure Tables client library for .NET - version 12.12.0.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/overview/azure/data.tables-readme?view=azure-dotnet [Accessed 04 September 2026].


### `OrderStatus.cs`

Code Attribution: The order status constants and lifecycle validation logic were implemented for the CoffeeNChill project to control the order progression from Received to Preparing, Ready and Collected, as described in the CLDV6212 POE brief (IIE, 2026).

### `ErrorResponse.cs`

Code Attribution: The HTTP error response model was informed by Microsoft's official Azure Functions HTTP trigger documentation.


Microsoft Learn (2025b). *Azure Functions HTTP trigger.* [Online] Available at: https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-http-webhook-trigger?tabs=python-v2%2Cisolated-process%2Cnodejsv4%2Cfunctionsv2&pivots=programming-language-csharp [Accessed 04 September 2026].


### `CreateOrderRequest.cs`

Code Attribution: Separating the request model from the queue message model was based on the lecturer-provided StudentQuery examples (IIE, 2026). The request fields were taken from the CLDV6212 POE brief (IIE, 2026).

### `OrderResponse.cs`

Code Attribution: DTO returned to clients so table internals (ETag, Timestamp) stay hidden and the comma-joined SKU text is turned back into a list. The entity-to-response mapping approach was based on Microsoft's official Azure Tables client library documentation.

Microsoft Learn (2026c). *Azure Tables client library for .NET - version 12.12.0.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/overview/azure/data.tables-readme?view=azure-dotnet [Accessed 04 September 2026].

### `OrderQueuedResponse.cs`

Code Attribution: The HTTP response DTO pattern was based on Microsoft's official Azure Functions HTTP trigger documentation.

Microsoft Learn (2025b). *Azure Functions HTTP trigger.* [Online] Available at: https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-http-webhook-trigger?tabs=python-v2%2Cisolated-process%2Cnodejsv4%2Cfunctionsv2&pivots=programming-language-csharp [Accessed 04 September 2026].

### `OrderStatusResponse.cs`

Code Attribution: The HTTP response DTO pattern was based on Microsoft's official Azure Functions HTTP trigger documentation. The response structure for the optional order status endpoint was based on the lecturer-provided course example (IIE, 2026).

Microsoft Learn (2025b). *Azure Functions HTTP trigger.* [Online] Available at: https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-http-webhook-trigger?tabs=python-v2%2Cisolated-process%2Cnodejsv4%2Cfunctionsv2&pivots=programming-language-csharp [Accessed 04 September 2026].

### `MenuItemRequest.cs`

Code Attribution: Separating the menu item request model from the table entity was based on the lecturer-provided StudentQuery and StudentQueryEntity course examples (IIE, 2026). Azure Table Storage entity handling follows Microsoft's official Azure Tables client library documentation.

Microsoft Learn (2026c). *Azure Tables client library for .NET - version 12.12.0.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/overview/azure/data.tables-readme?view=azure-dotnet [Accessed 04 September 2026].

### `MenuItemResponse.cs`

Code Attribution: Separating the menu item response model from the table entity was based on the model structure demonstrated in the lecturer-provided course examples (IIE, 2026).

### `OrderMapper.cs`

Code Attribution: The mapper pattern for converting between request, queue message and response objects was implemented for the CoffeeNChill project to keep the Azure Functions code short and easier to maintain, using the CLDV6212 POE brief (IIE, 2026).

### `StaffDocumentResponse.cs`

Code Attribution: The response DTO pattern for staff document information was based on Microsoft's official Azure Blob Storage documentation.

Microsoft Learn (2026e). *Introduction to Azure Blob Storage.* [Online] Available at: https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-introduction [Accessed 04 September 2026].

### `Dockerfile`

Code Attribution: Multi-stage build pattern adapted from the Azure Functions custom container documentation and the Dockerfile reference.

Microsoft Learn (2026b). *Work with containers and Azure Functions.* [Online] Available at: https://learn.microsoft.com/en-us/azure/azure-functions/functions-how-to-custom-container [Accessed 04 September 2026].

Docker (2026h). *Dockerfile reference.* [Online] Available at: https://docs.docker.com/reference/dockerfile/ [Accessed 09 October 2026].

### `docker-compose.yml`

Code Attribution: Structure adapted from the lecturer's Docker Compose notes (IIE, 2026); services and file layout from the Compose file reference, networks and volumes from the Docker documentation.

Docker (2026f). *Compose file reference.* [Online] Available at: https://docs.docker.com/reference/compose-file/ [Accessed 29 September 2026].

Docker (2026e). *Define and manage networks in Docker Compose.* [Online] Available at: https://docs.docker.com/reference/compose-file/networks/ [Accessed 29 September 2026].

Docker (2026g). *Define and manage volumes in Docker Compose.* [Online] Available at: https://docs.docker.com/reference/compose-file/volumes/ [Accessed 29 September 2026].

# REFERENCES


Docker, 2026a. *Build and push your first image.* [Online] Available at: https://docs.docker.com/get-started/introduction/build-and-push-first-image/ [Accessed 04 September 2026].

Docker, 2026b. *Docker Hub quickstart.* [Online] Available at: https://docs.docker.com/docker-hub/quickstart/ [Accessed 04 September 2026].

Docker, 2026c. *Push images to a repository.* [Online] Available at: https://docs.docker.com/docker-hub/repos/manage/hub-images/push/ [Accessed 04 September 2026].

Docker, 2026d. *Build, tag, and publish an image.* [Online] Available at: https://docs.docker.com/get-started/docker-concepts/building-images/build-tag-and-publish-an-image/ [Accessed 04 September 2026].

Docker, 2026e. *Define and manage networks in Docker Compose.* [Online] Available at: https://docs.docker.com/reference/compose-file/networks/ [Accessed 29 September 2026].

Docker, 2026f. *Compose file reference.* [Online] Available at: https://docs.docker.com/reference/compose-file/ [Accessed 29 September 2026].

Docker, 2026g. *Define and manage volumes in Docker Compose.* [Online] Available at: https://docs.docker.com/reference/compose-file/volumes/ [Accessed 29 September 2026].

Docker, 2026h. *Dockerfile reference.* [Online] Available at: https://docs.docker.com/reference/dockerfile/ [Accessed 09 October 2026].

IIE, 2026. *CLDV6212 lecturer course examples, notes and POE brief (Parts 1 and 2): Azure Functions, Table and Blob Storage, queue triggers and Docker Compose.* [Lecture notes]. Cloud Development B. The Independent Institute of Education.

Microsoft Learn, 2022. *What is Azure Table storage?* [Online] Available at: https://learn.microsoft.com/en-us/azure/storage/tables/table-storage-overview [Accessed 04 September 2026].

Microsoft Learn, 2025a. *Install and run Azurite emulator.* [Online] Available at: https://learn.microsoft.com/en-us/azure/storage/common/storage-install-azurite [Accessed 04 September 2026].

Microsoft Learn, 2025b. *Azure Functions HTTP trigger.* [Online] Available at: https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-http-webhook-trigger?tabs=python-v2%2Cisolated-process%2Cnodejsv4%2Cfunctionsv2&pivots=programming-language-csharp [Accessed 04 September 2026].

Microsoft Learn, 2026a. *Create your first containerized Azure Functions.* [Online] Available at: https://learn.microsoft.com/en-us/azure/azure-functions/functions-deploy-container [Accessed 04 September 2026].

Microsoft Learn, 2026b. *Work with containers and Azure Functions.* [Online] Available at: https://learn.microsoft.com/en-us/azure/azure-functions/functions-how-to-custom-container [Accessed 04 September 2026].

Microsoft Learn, 2026c. *Azure Tables client library for .NET - version 12.12.0.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/overview/azure/data.tables-readme?view=azure-dotnet [Accessed 04 September 2026].

Microsoft Learn, 2026d. *Download a blob with .NET.* [Online] Available at: https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blob-download [Accessed 04 September 2026].

Microsoft Learn, 2026e. *Introduction to Azure Blob Storage.* [Online] Available at: https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-introduction [Accessed 04 September 2026].

Microsoft Learn, 2026f. *MultipartReader Class.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.webutilities.multipartreader?view=aspnetcore-10.0 [Accessed 04 September 2026].

Microsoft Learn, 2026g. *Azure Queue storage trigger for Azure Functions.* [Online] Available at: https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-storage-queue-trigger [Accessed 29 September 2026].

Microsoft Learn, 2026h. *Conditional operator.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/conditional-operator [Accessed 30 September 2026].

Microsoft Learn, 2026i. *String.Join Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.string.join?view=net-10.0 [Accessed 30 September 2026].

Microsoft Learn, 2026j. *Task.Delay Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.threading.tasks.task.delay?view=net-10.0 [Accessed 01 October 2026].

Microsoft Learn, 2026k. *TableClient.UpsertEntityAsync Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/azure.data.tables.tableclient.upsertentityasync?view=azure-dotnet [Accessed 01 October 2026].

Microsoft Learn, 2026l. *Poison messages in Azure Functions.* [Online] Available at: https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-storage-queue-trigger#poison-messages [Accessed 01 October 2026].

Microsoft Learn, 2026m. *Environment.GetEnvironmentVariable Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.environment.getenvironmentvariable?view=net-10.0 [Accessed 01 October 2026].

Microsoft Learn, 2026n. *Int32.TryParse Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.int32.tryparse?view=net-10.0 [Accessed 05 October 2026].

Microsoft Learn, 2026o. *Convert.FromBase64String Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.convert.frombase64string?view=net-10.0 [Accessed 08 October 2026].

Microsoft Learn, 2026p. *Encoding.GetString Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.text.encoding.getstring?view=net-10.0 [Accessed 08 October 2026].

Microsoft Learn, 2026q. *Azure Queue Storage client library for .NET.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/overview/azure/storage.queues-readme?view=azure-dotnet [Accessed 04 September 2026].

Microsoft Learn, 2026r. *RequestFailedException Class.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/azure.requestfailedexception?view=azure-dotnet [Accessed 01 October 2026].

Microsoft Learn, 2026s. *JsonSerializer.Deserialize Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.text.json.jsonserializer.deserialize?view=net-10.0 [Accessed 05 October 2026].

Microsoft Learn, 2026t. *StreamReader.ReadToEndAsync Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.io.streamreader.readtoendasync?view=net-10.0 [Accessed 05 October 2026].

Microsoft Learn, 2026u. *BlobHttpHeaders Class.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/azure.storage.blobs.models.blobhttpheaders?view=azure-dotnet [Accessed 04 September 2026].

Microsoft Learn, 2026v. *BlobUploadOptions.HttpHeaders Property.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/azure.storage.blobs.models.blobuploadoptions.httpheaders?view=azure-dotnet [Accessed 04 September 2026].

Microsoft Learn, 2026w. *Convert.ToBase64String Method.* [Online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.convert.tobase64string?view=net-10.0 [Accessed 09 October 2026].

Mrzyglod, K. and Camden, R., 2020. *Azure for Developers: Implement rich Azure PaaS ecosystems using containers, serverless services, and storage solutions.* 2nd ed. Birmingham: Packt Publishing.

Postman, 2025. *Write scripts to test API response data in Postman.* [Online] Available at: https://learning.postman.com/docs/tests-and-scripts/write-scripts/test-scripts/ [Accessed 01 September 2026].